NAZARBAYEV UNIVERSITY

# Programming for Engineers

## Detailed Reading Notes

## Week 13: Python and Programming Beyond Python

**Yanwei Wang**

School of Engineering

Nazarbayev University

Fall 2026

**Keep the problem and evidence.**
**Change the language deliberately.**

SCHOOL OF ENGINEERING

# About these notes

These notes expand the **87-page Week 13 presentation, Python and Programming Beyond Python**, previously shared for ENG 101. They retain the three-lecture sequence, named examples, engineering contracts, activities, and optional reference material in that presentation. The goal is not to complete three additional programming courses. It is to learn how familiar reasoning survives a change of syntax, data representation, and execution environment.

**Course information**

**Course:** ENG 101 — Programming for Engineers; lecture section 2L; 6 ECTS.
**Instructor:** Dr. Yanwei Wang, School of Engineering, Nazarbayev University.
**Term:** Fall 2026.
**Session 13:** Python usage and other programming languages. The source scope identifies CLOs 1, 2, 3, and 5. Its "C+" is explicitly interpreted as **C++**; the optional C program remains a separate language example.

*Lecture-slide alignment: Week 13 source PDF, pp. 1–3.*

## A reading route

| Lecture | Central question |
|---|---|
| 37: Python in engineering and transferable ideas | What must remain true when a calculation moves into another language? |
| 38: MATLAB for Python users | Which values, positions, array operations, and numerical updates are actually equivalent? |
| 39: C++ and Java: read, translate, and test | How do explicit types, collections, and object operations express the same documented task? |

Read the Python baseline first. Work out an independent answer using small values. Then read the unfamiliar code from the calculation outward: identify the inputs, operation, returned value, and only then the surrounding language structure. Write down any change in indexing, shape, copying, or failure behavior before declaring a translation equivalent.

**Key idea**

**Keep the problem and evidence; change the language deliberately.** A program that compiles can still use the wrong units, select the wrong sample, change an inclusive boundary, or corrupt state before rejecting an update.

## Code, comments, and execution status

Every code listing has comments in its language: `#` in Python and shell examples, `%` in MATLAB, `//` in C++ and Java, `/* ... */` in C, and `-` in SQL. Examples labelled *definitions* or *excerpts* belong inside their supplied module, header, class, or driver; they are not all stand-alone console programs. Numbered Python examples are in `examples/python/`. Other language folders contain the matching native files and their supporting definitions.

**MATLAB examples have not been executed in MATLAB in this preparation environment.** Their source has been reviewed against the interfaces cited in the slides and current official documentation. Expected MATLAB results are derived from the displayed expressions and independent calculations, never described as captured MATLAB output. Native MATLAB checks are supplied for a later run. No Octave run is used as a substitute for MATLAB execution.

Python, C++, Java, and the optional C example have separate execution records for this reading package. A program run using a headless plotting backend checks the plotting code path, not the behavior of a student's graphical desktop. Exact whitespace in printed arrays, help text, and tiny floating-point residuals can vary with the software version.

## Source content and worked additions

The source PDF, its lecture LaTeX, source map, instructor explanations, and self-contained Python companion were available. The companion's five principal Python definitions were recovered. The complete historical C++, Java, and MATLAB modules, the original cross-language test scripts, and their raw evidence files were *not* present in the recovered materials. Accordingly, complete wrappers, missing implementations, additional tests, activity answers, and cumulative exercises are explicitly labelled as **worked completions** rather than silently presented as original files.

The historical test totals printed on source page 73 are reported only as statements made by the slides. They are not reused as this document's verification counts. Newly executed tests and regenerated cooling values are recorded separately. The original cooling CSV was unavailable; the curve is regenerated from the recovered recurrence and the exact formula stated on source page 50.

**A common domain does not mean identical type systems**

The shared examples use small, finite real numbers representable in the selected floating types. Python explicitly rejects Boolean values in its numeric helper; a C++ or Java `double` interface follows different native conversion rules. Agreement is tested on the declared common domain, not claimed for every possible object, numeric magnitude, or language input.

## Study and assessment boundaries

Predict before running, change one input, explain the changed result, and rerun the retained tests. Activities and their worked answers support independent study; no group partner or timed session is needed. Optional matrix solving, SQL, C, and awareness of other tools remain optional. These notes do not introduce a new assignment, deadline, software-purchase requirement, installation requirement, or permission to use AI in assessed work. Follow the applicable Moodle instructions.

Page references in these notes mean **physical pages of the 87-page source PDF**, not its smaller footer frame numbers. Appendix A.3 maps every source page to a reading section. The source PDF is the primary basis; documentary checks and explanatory additions are identified where they qualify its brief comparisons.

# Contents

**About these notes** **2**

**1. Python in Engineering and Transferable Ideas** **6**
1.1. Start with a familiar engineering calculation . . . 6
1.2. Language, library, editor, and runtime . . . 7
1.3. A complete small sensor workflow . . . 8
1.4. The same formula can have a different surface . . . 9
1.5. Execution models and reproducibility . . . 10
1.6. Types, syntax, and physical meaning . . . 11
1.7. Two compact but important operator traps . . . 12
1.8. Translate the contract and the evidence . . . 13
1.9. Choose and learn a tool deliberately . . . 14
1.10. Lecture 37 summary and exit-ticket answers . . . 15

**2. MATLAB for Python Users** **16**
2.1. Begin with the object, not the operator . . . 16
2.2. Scalar statements and intentional output . . . 17
2.3. A numeric array has both values and shape . . . 17
2.4. Translate positions and endpoint rules . . . 18
2.5. Elementwise and matrix operations ask different questions . . . 19
2.6. Masks and text comparisons . . . 21
2.7. Conditions and loops retain their contracts . . . 21
2.8. Translate functions and their tests together . . . 22
2.9. A plot has a data and model contract . . . 24
2.10. Cooling: preserve the discrete update . . . 25
2.11. Agreement, discretization error, and physical evidence . . . 27
2.12. Debug a mixed-language translation one defect at a time . . . 29
2.13. Lecture 38 summary and exit-ticket answers . . . 30

**3. C++ and Java: Read, Translate, and Test** **31**
3.1. Read a complete C++ program from the calculation outward . . . 31
3.2. Java places the same calculation inside a class . . . 32
3.3. Declared types do not settle numerical meaning . . . 33
3.4. Loops, functions, and methods retain the same task . . . 35
3.5. Collections: positions, lengths, and failure behavior . . . 36
3.6. String content and object identity . . . 37
3.7. The same assignment can preserve or share storage . . . 38
3.8. Translate a state-change contract . . . 38
3.9. Test rejection and the surviving state separately . . . 40
3.10. Exchange data through a documented interface . . . 43
3.11. Worked activity: transfer a first-threshold search . . . 43
3.12. Review a translation and close the week . . . 45

**4. Optional Connections and Reference Examples** **46**
4.1. Solve the same pair of linear equations . . . 46
4.2. SQL requests a result from structured records . . . 47
4.3. C is not another name for C++ . . . 48
4.4. Other tools: recognition rather than an installation list . . . 49

4.5. Use the supplied package as a reproducible study resource . . . . . 49

**5. Cumulative Practice and Worked Solutions** **50**
5.1. Practice questions . . . . . 50
5.2. Worked solutions . . . . . 52

**6. Quick Reference, Glossary, and Study Checklist** **54**
6.1. Cross-language recognition table . . . . . 54
6.2. Glossary . . . . . 54
6.3. A cross-language review checklist . . . . . 56

**A. Source basis, coverage, and verification** **57**
A.1. What was preserved and what was completed . . . . . 57
A.2. Execution record for this package . . . . . 57
A.3. Slide-to-reading coverage . . . . . 58

**Bibliography** **60**

# 1 Python in Engineering and Transferable Ideas

*Lecture-slide alignment: Week 13 source PDF, pp. 4–28; Lecture 37.*

**Learning outcomes**

After studying this chapter, you should be able to explain Python's role in an engineering workflow; distinguish a language, library, editor, runtime, and toolchain; identify concepts that transfer; diagnose numeric and syntactic translation traps; and specify evidence before accepting a translated program.

## 1.1. Start with a familiar engineering calculation

*Lecture-slide alignment: Week 13 source PDF, pp. 5–7.*

Consider a uniform axial force of 5000 N acting on an area of 0.002 $m^2$. The model is $\sigma = F/A$. Before opening an editor, calculate

$$\sigma = \frac{5000}{0.002} = 2{,}500{,}000\,\text{Pa} = 2.50\,\text{MPa}.$$

The two unit conversions are not interchangeable: force divided by square metres gives pascals, whereas the final report deliberately displays megapascals. This distinction is part of the input–output contract, not an incidental formatting choice.

```python
# Source slide 6: uniform axial stress, with force in N and area in m^2.
force_n = 5000.0
area_m2 = 0.002
stress_pa = force_n / area_m2  # N/m^2 gives Pa.
# Convert the result for display only; 1 MPa = 1,000,000 Pa.
print(f"Stress = {stress_pa / 1e6:.2f} MPa")
```

**Code 1.1:** Python baseline: calculate in Pa and display in MPa.

```
Stress = 2.50 MPa
```

The assignments give names to the input quantities. The division applies the engineering model. The expression inside the f-string divides by $10^6$ only for display, and `.2f` asks for two digits after the decimal point. It does not improve measurement accuracy. A translation may rename variables or use a different output function, but it must not accidentally reinterpret the area as square millimetres or label an unconverted number as MPa.

**Warm-up answer**

The inputs are force in N and area in $m^2$; the stored result is Pa and the report is MPa. Another language must preserve the quantities, formula, valid domain, and output meaning. The constant-area, uniform-stress model is still a simplified model; moving it into a compiled program does not validate an actual component.

The knowledge accumulated in Weeks 1–12 is therefore the starting point, not something to discard. Variables still represent quantities or records. Conditions still encode decision rules. A loop still needs a progress argument. Functions and classes still need coherent responsibilities. Tests still need independently justified expectations. A search still has a contract, and a full scan still visits its candidates even when expressed with different punctuation.

## 1.2. Language, library, editor, and runtime

*Lecture-slide alignment: Week 13 source PDF, pp. 8–9, 12–13.*

A **language** supplies rules for expressing computations. A **library** supplies reusable implementations through named interfaces. An **editor** or integrated development environment is a workspace for creating and inspecting source code. A **runtime** executes a program, and a **toolchain** can include compilers, linkers, runtimes, and related build tools.

| Layer | Examples in the slides | Question it answers |
|---|---|---|
| Language | Python, C++, Java | How can the algorithm be written? |
| Built-in constructs | Lists, loops, functions | Which core forms represent data and control flow? |
| Standard library | `math`, `csv`, `pathlib` | Which supporting tools come with this Python environment? |
| Third-party library | NumPy, Matplotlib | Which additional numeric or plotting interface is being used? |
| Editor / IDE | VS Code | Where is the source edited and inspected? |
| Runtime / toolchain | Interpreter, JDK, C++ compiler | What actually builds or runs the source? |
| Your own code | Checked sensor functions | Where are the problem-specific rules implemented? |

MATLAB names both a programming language and a computing environment in this comparison. NumPy is not a synonym for Python, and a Python list is not a NumPy array. Installing an editor alone does not install every interpreter, compiler, or library that the code may need. These distinctions help diagnose a failed run: an unavailable import is not the same problem as an incorrect engineering expression.

The workflow diagram on source page 9 places a calculation inside a larger process. The diagram below preserves that order without implying that every box needs a new software package.

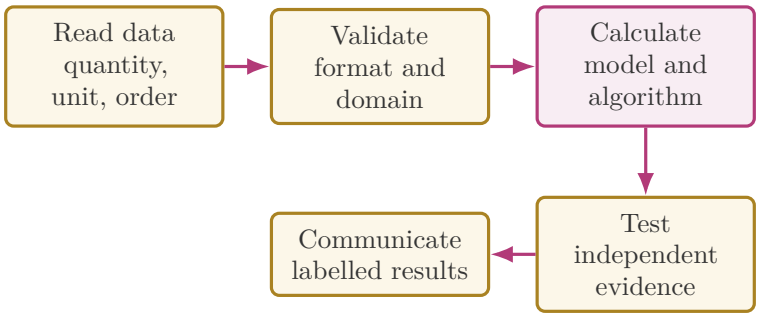



**Figure 1.1.:** The calculation is one stage of an engineering workflow.

Source basis: source page 9; engineering workflow. Diagram redrawn for these notes.

The input must retain quantity, unit, and ordering information. Validation checks format and domain. Calculation uses a stated model and algorithm. Testing checks independently expected behavior. Communication reports the result with its assumptions and limits. Python can coordinate this sequence, but the programmer must decide what each stage promises.

### 1.2.1. Read an unfamiliar call through its interface

```python
# Source slide 13: inspect the domain and result of an unfamiliar call.
import math
root = math.sqrt(9.0)  # Hand-checkable input: 3 squared is 9.
print(root)
help(math.sqrt)  # Documentation text can differ between Python versions.
```

**Code 1.2:** Inspect a hand-checkable call and request its documentation.

The numerical result of `math.sqrt(9.0)` is `3.0`. The following `help` call asks the installed environment for documentation; its full output is not a fixed numerical reference. Before applying a new function, locate accepted types and domain, returned type and shape, documented units if any, and failure behavior. A memorable function name is not a complete contract.

**Engineering check**

A square-root result does not automatically inherit an engineering unit from a bare number. If the input represents an area, the physical interpretation of its square root is the programmer's responsibility. Likewise, a new library does not know which of two equal-length arrays contains forces and which contains areas.

## 1.3. A complete small sensor workflow

*Lecture-slide alignment: Week 13 source PDF, pp. 10–11.*

The source uses five *synthetic* readings, not observations from a real sensor:

$$[19.5,\ 20.0,\ 25.0,\ 30.0,\ 30.5]\ ^\circ\text{C}.$$

The normal interval is inclusive: $20 \leq T \leq 30$. The task is to preserve warning locations while also calculating a mean.

```python
# Synthetic data from slide 10; this is not a hardware measurement.
readings_c = [19.5, 20.0, 25.0, 30.0, 30.5]
low_c, high_c = 20.0, 30.0  # Both endpoints belong to the normal range.
warnings = []  # Keep both the original position and the warning value.
for index, value_c in enumerate(readings_c):
    if value_c < low_c or value_c > high_c:
        warnings.append((index, value_c))
# Nonempty finite data are assumed in this introductory workflow.
mean_c = sum(readings_c) / len(readings_c)
print(f"Mean = {mean_c:.2f} deg C")
print("Warnings:", warnings)
print("Raw readings:", readings_c)  # The raw list was not mutated.
```

**Code 1.3:** Retain original indices, individual warnings, and raw data.

```
Mean = 25.00 deg C
Warnings: [(0, 19.5), (4, 30.5)]
Raw readings: [19.5, 20.0, 25.0, 30.0, 30.5]
```

`enumerate` supplies a zero-based index and its associated reading together. The `or` condition detects either side of the interval. Each appended tuple records both where the warning occurred and its value. The loop never assigns to `readings_c`, so the supplied sequence is preserved.

| Index | Temperature | Condition | Effect |
|---|---|---|---|
| 0 | 19.5 | Below 20 | Append `(0, 19.5)`. |
| 1 | 20.0 | At included lower limit | No warning. |
| 2 | 25.0 | Inside interval | No warning. |
| 3 | 30.0 | At included upper limit | No warning. |
| 4 | 30.5 | Above 30 | Append `(4, 30.5)`. |

The independent total is 125.0, so the mean is $125.0/5 = 25.0\,^{\circ}\mathrm{C}$. That aggregate does not erase the two out-of-limit values. Reporting only the mean would answer a weaker question. Sorting the raw data before assigning indices would also change the meaning of a warning location if positions represent acquisition order.

**Two warnings are not inconsistent with one high reading**

The two-sided normal-range task has two warnings. The separate `count_above(values, 30.0)` contract counts only values strictly above 30 and therefore returns one. These are different questions. Keep the boundary rule and the requested result visible when translating them.

The mean expression assumes nonempty finite data. A production function could reject empty or nonfinite input explicitly, but that is not performed by this small source script. Its simple form must not be described as a complete sensor-data validator.

## 1.4. The same formula can have a different surface

*Lecture-slide alignment: Week 13 source PDF, p. 14.*

For an ideal resistance $R > 0$, Ohm's law gives $I = V/R$. Both of the following examples use $12\,\mathrm{V}$ and $4\,\Omega$, so both represent $3\,\mathrm{A}$.

```python
# Source slide 14: chosen units are volts, ohms, and amperes.
voltage_v = 12.0
resistance_ohm = 4.0  # This small example assumes positive resistance.
current_a = voltage_v / resistance_ohm
print(current_a)  # Expected numerical result: 3 A.
```

**Code 1.4:** Python: scalar division with named physical quantities.

```m
% Source slide 14: the same scalar quantities as in Python.
voltage_v = 12.0;
resistance_ohm = 4.0;  % Positive resistance is assumed.
current_a = voltage_v / resistance_ohm;
disp(current_a);  % Expected numerical result: 3 A.
```

**Code 1.5:** MATLAB: the same scalar calculation and a different output call.

Expected result, derived from the code and stated model; not captured MATLAB output: the scalar value is 3 A; display spacing may differ from Python.

In MATLAB, `%` introduces a comment. A semicolon suppresses automatic display of an assignment result; it does not suppress an intentional `disp` or `fprintf` call. Python normally uses line breaks to separate statements. These are syntax differences, not changes to the engineering formula.

## 1.5. Execution models and reproducibility

*Lecture-slide alignment: Week 13 source PDF, pp. 15, 23.*

The source shows three common execution routes. They describe practical toolchains, not immutable categories that every implementation of a language must follow.

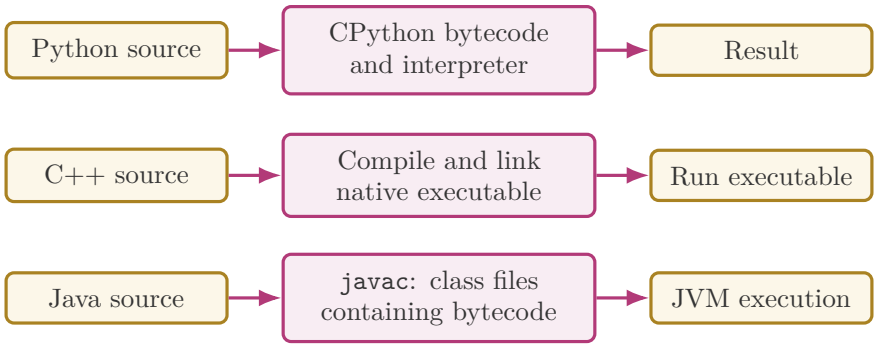



**Figure 1.2.:** Common source-to-execution routes shown in the lecture.

Source basis: source page 15; common execution paths. Diagram redrawn for these notes.

In the CPython route, source is compiled to bytecode and executed by the interpreter. The usual C++ route shown here compiles and links source to a native executable, which is then run. The Java route uses `javac` to create class files containing bytecode, then uses the Java runtime to execute the program. A Java implementation may interpret and compile bytecode during execution; the purpose of the diagram is to separate build tools from source editing, not to catalogue every implementation strategy.[4, 9]

Reproducibility requires the source, its inputs, the actual compiler or interpreter version, any libraries, relevant compiler flags, and the command used to run it. A screenshot alone does not let another person reproduce a result. A language name also does not establish execution speed: a Python loop and a call into an optimized numeric library are different implementations with different work.

**Key idea**

An execution error may concern a missing tool, an invalid source program, a runtime exception, or a numerically incorrect result. Diagnose which layer failed before changing the engineering formula.

## 1.6. Types, syntax, and physical meaning

*Lecture-slide alignment: Week 13 source PDF, pp. 16–18.*

Dynamic typing does not mean that Python values lack types. In the source example, the name `value` first refers to an integer and is then rebound to a string. In the C++ or Java example, a variable declared `int` cannot later be assigned the string literal `"three"`. Static checking can detect some type mismatches, but it cannot infer the programmer's intended unit system.

```python
# Slide 16: reassignment changes which object a Python name refers to.
value = 3
value = "three"
print(type(value).__name__)
# Slides 19-20: inspect the operation before translating the symbol.
print(7 / 2)  # True division: 3.5 in Python 3.
print(7.0 / 2.0)
print(3 ** 2)  # Exponentiation: 9.
print(3 ^ 2)  # Integer exclusive-or: 1, not a square.
```

**Code 1.6:** Rebinding, division, power, and XOR in the Python baseline.

```
str
3.5
3.5
9
1
```

The last four printed results will be examined below. The first printed result, `str`, confirms that the name now refers to text. The integer object did not somehow become a string object.

| Idea | Python | MATLAB | C++ / Java |
|---|---|---|---|
| Comment | `#` | `%` | `//` |
| Ordinary statement ending | Newline | `;` suppresses echo | `;` |
| Block structure | Colon and indentation | Matching `end` | Braces `{ ... }` |
| Numeric equality | `==` | `==` | `==` |
| Numeric inequality | `!=` | `~=` | `!=` |

Recognition table from source page 18. Text, arrays, and objects need additional rules; this table is not a universal operator-translation recipe.

### 1.6.1. A typed program can still mislabel a unit

The source deliberately names $F/A$ as `stressPa` when $F = 10000\,\mathrm{N}$ and $A = 500\,\mathrm{mm}^2$. The calculation returns 20, but

$$\frac{1\,\mathrm{N}}{1\,\mathrm{mm}^2} = \frac{1\,\mathrm{N}}{10^{-6}\,\mathrm{m}^2} = 10^6\,\mathrm{Pa} = 1\,\mathrm{MPa}.$$

Thus 20 means MPa, not Pa. Either keep the area in square millimetres and report MPa, or convert the area to square metres before reporting Pa. A declaration of type `double` does not detect this factor-of-a-million interpretation error.

## 1.7. Two compact but important operator traps

*Lecture-slide alignment: Week 13 source PDF, pp. 19–20.*

### 1.7.1. Division happens before assignment

| Expression | Python 3 | MATLAB doubles | C++ / Java |
|---|---|---|---|
| `7 / 2` | 3.5 | 3.5 | 3 (integer operands) |
| `7.0 / 2.0` | 3.5 | 3.5 | 3.5 |

In `double lost = 7 / 2;`, integer division is evaluated first. The integer result 3 is then converted to a floating value. The destination can store a fractional value, but it cannot recover a fraction discarded earlier. Making an operand floating *before* division changes the evaluated expression.[11, 10]

**Stop, predict, and try it**

Predict the difference between a count of seven samples split into two groups and the numerical ratio 7/2. Are you requesting a whole-number quotient, an exact ratio, or a mean group size? Choose the operation from the required meaning, not from the variable name alone.

### 1.7.2. The caret symbol does not have a portable meaning

For integer literals, Python, C++, and Java use `^` for bitwise exclusive-or. Hence `3 ^ 2` produces 1 in these examples, whereas MATLAB's scalar `3 ^ 2` produces 9. Python exponentiation uses `**`; squaring with `x * x` is an explicit alternative in the C++ and Java examples. MATLAB elementwise array powers use `.^`, which will be developed in Chapter 2.

The problem is semantic, not merely visual. A copied expression may run and return a completely valid integer while answering the wrong question. A known-answer test such as "3 squared must be 9" catches the mistake immediately.

## 1.8. Translate the contract and the evidence

*Lecture-slide alignment: Week 13 source PDF, pp. 21–22.*

The later shared stress function intentionally has a *different area unit* from the opening scalar script: it receives area in $\mathrm{mm}^2$ and returns stress in MPa. This is a documented interface, not an unexplained numerical discrepancy.

**Function contract**

**Inputs:** finite real scalars, force in N and area in $\mathrm{mm}^2$.
**Domain:** area strictly positive; force may be zero or signed.
**Output:** uniform axial stress in MPa.
**Failure:** reject invalid area; do not invent a replacement stress.
**Known answer:** $10000\,\mathrm{N}/500\,\mathrm{mm}^2 = 20\,\mathrm{MPa}$.

```python
# Recovered Week 13 companion definition; see the stated input contract.
def finite_number(value):
    """Accept built-in finite real numbers, excluding bool."""
    if isinstance(value, bool) or not isinstance(value, (int, float)):
        raise TypeError("a real number is required")
    if not math.isfinite(value):
        raise ValueError("the number must be finite")
    return float(value)
```

**Code 1.7:** Recovered Python helper: a chosen finite-number contract.

The first condition deliberately excludes `bool`, even though Boolean values have an integer relationship in Python. Text is not automatically converted. The second condition rejects NaN and infinity. The returned float supplies a consistent numeric representation to the calculation. These examples use small values representable as finite floats; arbitrary-size integer conversion is not the task being taught.

```python
# Recovered Week 13 companion definition; see the stated input contract.
def stress_mpa(force_n, area_mm2):
    """Return uniform axial stress in MPa; area must be positive."""
    force_n = finite_number(force_n)
    area_mm2 = finite_number(area_mm2)
    if area_mm2 <= 0.0:
        raise ValueError("area must be positive")
    # One N/mm^2 is one MPa; no factor is silently omitted.
    result = force_n / area_mm2
    if not math.isfinite(result):
        raise ValueError("result is outside the finite range")
    return result
```

**Code 1.8:** Recovered stress function: preserve the unit contract and reject invalid area.

The independent tests should include a zero force, a signed negative force under the chosen convention, a positive area, and rejection of zero area. The strict high-count test uses $[29.5, 30.0, 30.5]$ with a limit of 30; exactly one value qualifies. The object test asks not only whether $-1$ was rejected, but also whether the previous valid value 12 survived.

| Shared check | Independent expectation |
|---|---|
| Stress: 10000 N, 500 $mm^2$ | 20 MPa |
| Strictly above 30 in the five-reading dataset | One value |
| Mean of the five-reading dataset | 25 °C |
| Propose a sensor update to −1 | Rejection |
| Read the state after that rejection | 12, not −1 |

The source slide reports these results in Python, C++, and Java. The note package supplies new drivers for the same common checks. The corresponding full native definitions are printed later. Agreement among implementations supports translation equivalence over those tests; it does not prove an engineering model universally correct. Several translations can repeat the same wrong formula or share the same inadequate test case.

## 1.9. Choose and learn a tool deliberately

*Lecture-slide alignment: Week 13 source PDF, pp. 24–27.*

Language choice begins with constraints. What must the program run on or connect to? Which libraries and interfaces already exist? What accuracy, performance, and memory evidence is required? Who will maintain the code, and how will someone else reproduce it? These are stronger questions than asking which language is universally best.

### 1.9.1. Worked answers to the tool-choice activity

For **batch-processing sensor CSV files**, investigate the file schema, quoting rules, unit fields, validation needs, and repeatable reporting. Python is a candidate workflow tool in the source framing, but the argument should name its fit to the actual files and team rather than simply assert popularity.

For **extending an existing controls model**, inspect the environment already used, compatible interfaces, and required toolboxes before considering a rewrite. A working MATLAB model is not automatically improved by moving it elsewhere. No campus licence entitlement is implied by this scenario.

For **a constrained device**, investigate its supported compiler, memory budget, target interfaces, and measured timing. A desktop demonstration does not establish that code will fit or meet a deadline on another device. For **an existing Java service**, existing build tools, data interfaces, deployment conventions, and regression tests may dominate the decision. These four scenarios are hypothetical, not new course projects.

### 1.9.2. Use AI-assisted translation as a checked draft

The slides suggest asking for a translation that preserves units, valid inputs, strict or inclusive limits, returned results, and failure behavior, with an explanation of every changed construct. The student must still inspect documentation and run independent tests. Sensitive data should not be placed in an external prompt, and the applicable task's assistance policy remains authoritative.

A practical learning sequence is to run the smallest valid target-language program, translate one labelled formula, add a condition and a loop, define a small function, and learn the main collection's indexing rules. Rebuild one familiar example and reuse its tests before adding

advanced features. This keeps each new syntactic idea connected to a known computational purpose.

## 1.10. Lecture 37 summary and exit-ticket answers

*Lecture-slide alignment: Week 13 source PDF, p. 28.*

**Section summary**

Distinguish the engineering contract from its surface syntax and execution toolchain. Check operand types before interpreting division, do not assume `^` means exponentiation, and keep independent tests for units, boundaries, and failure behavior. Preserve the problem and the evidence, not necessarily the spelling of every identifier.

**Why can a floating destination lose the fraction in `7 / 2`?** Because the integer expression is evaluated before its result is assigned. The conversion to `double` occurs too late to restore the missing fraction.

**What must a stress translation preserve?** Force and area meanings and units, the chosen valid domain, the uniform-stress formula, the output unit, and the response to invalid area. Zero and signed-force tests must follow the stated contract rather than an unstated physical assumption.

**Why is agreement insufficient for complete validation?** It checks consistency on selected inputs. Both implementations may share a defect or use a model inappropriate for the real component. At least one independent answer and explicit limitations are still needed.

# 2 MATLAB for Python Users

*Lecture-slide alignment: Week 13 source PDF, pp. 29–53; Lecture 38.*

**Learning outcomes**

After studying this chapter, you should be able to read MATLAB scalar calculations and formatted output; translate positions, ranges, and array operations deliberately; recognize conditions, loops, and function interfaces; compare Python and MATLAB cooling updates; and check numerical results and axis meanings.

## 2.1. Begin with the object, not the operator

*Lecture-slide alignment: Week 13 source PDF, pp. 30–31.*

The expression `values * 2` is not enough to determine what will happen. Its meaning depends on the type of `values`. In the source warm-up, an ordinary list repeats its sequence, while a NumPy array multiplies its numerical entries.

```
# Slide 31: the list repeats its elements; the array scales each number.
values = [1, 2, 3]
print(values * 2)
import numpy as np  # Third-party numeric array library.
values = np.array([1.0, 2.0, 3.0])
print(values * 2)
```

**Code 2.1:** The same multiplication symbol acts on two different types.

```
[1, 2, 3, 1, 2, 3]
[2. 4. 6.]
```

The list result contains six elements; it does not contain the three doubled values. The array result retains three numerical entries, each twice its original value. This is the first important bridge to MATLAB: an ordinary numeric MATLAB array is a closer counterpart to a NumPy array than to a general Python list. The result of an operation must be understood through its interface and data type.

**Warm-up answer**

The list prints `[1, 2, 3, 1, 2, 3]`. The numeric array contains $2, 4, 6$. The apparent similarity of the expressions does not make the tasks equivalent. When porting a program, name the source and target data types before deciding how an operator should translate.

## 2.2. Scalar statements and intentional output

*Lecture-slide alignment: Week 13 source PDF, p. 32.*

The MATLAB stress example uses exactly the same 5000 N force and 0.002 $m^2$ area as the Python baseline. The result is again 2.50 MPa.

```
% Source slide 32: force in N and area in m^2 give stress in Pa.
force_n = 5000.0;
area_m2 = 0.002;
stress_pa = force_n / area_m2;
% Suppress automatic echo, then print an intentional labelled report.
fprintf('Stress = %.2f MPa\n', stress_pa / 1e6);
```

**Code 2.2:** MATLAB scalar calculation with a labelled formatted result.

Expected result, derived from the code and stated model; not captured MATLAB output: `Stress = 2.50 MPa`.

A percent sign begins a comment outside a quoted string. Within the `fprintf` format string, `%.2f` is a formatting instruction: display a floating-point value to two decimal places. The sequence `\n` in the format string requests a newline. The semicolon after a statement prevents an automatic echo; the explicit `fprintf` call still produces output.

The names include units to help the reader, but MATLAB does not enforce those units just because the names include them. The formula and the displayed unit must still be checked independently.

## 2.3. A numeric array has both values and shape

*Lecture-slide alignment: Week 13 source PDF, p. 33.*

The row `[1 2 3]` and the column `[1; 2; 3]` contain the same three numbers in different shapes. Spaces separate entries within a row; semicolons inside brackets separate rows. The function `size` reports dimensions. The source row has shape $1 \times 3$, while the column has shape $3 \times 1$.

Shape matters because an operation may combine corresponding entries, form a matrix product, or expand compatible operands. The fact that each input contains three numbers does not establish that an operation performs three pairwise calculations. Before combining two sets of measurements, ask whether the first entry belongs with the first entry, which dimension represents samples, and which dimension represents channels.

**Shape can change a valid result, not only cause an error**

Official MATLAB documentation describes implicit expansion for compatible array sizes. Therefore a row–column mismatch may create a larger array instead of producing an error. For the source's pairwise force/area example, deliberately use matching vector shapes. This is a qualification of the brief shape warning, not a new array-programming unit.[6]

## 2.4. Translate positions and endpoint rules

*Lecture-slide alignment: Week 13 source PDF, pp. 34–36.*

For a Python list, the first position is 0; for a MATLAB vector, it is 1. The second through fourth measurements must remain the second through fourth measurements after translation, even though their written indices change.

| Meaning | Python list | MATLAB vector |
|---|---|---|
| First value | `x[0]` | `x(1)` |
| Last value | `x[-1]` | `x(end)` |
| Second through fourth | `x[1:4]` | `x(2:4)` |
| Number of values | `len(x)` | `numel(x)` |

The Python slice stop is excluded. MATLAB's colon progression includes its upper endpoint only when that progression reaches it. These are two differences: the starting index convention and the endpoint convention. Mechanically adding one to both ends of a Python slice does not translate it correctly.

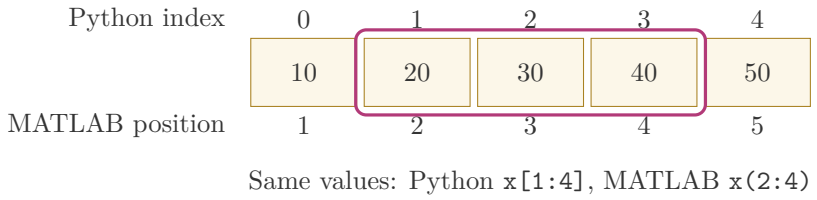



**Figure 2.1.:** Different index conventions can select the same three measurements.

Source basis: source pages 34–35; selection of the same physical measurements. Diagram redrawn for these notes.

```
% Source slides 33-35: shape and position are separate properties.
row = [1 2 3];        % One row, three columns.
column = [1; 2; 3];   % Three rows, one column.
disp(size(row));
disp(size(column));
x = [10 20 30 40 50];  % Complete the vector used in the selection exercise.
disp(x(1:4));  % Proposed A: 10, 20, 30, 40 (not the required selection).
disp(x(2:4));  % Proposed B: 20, 30, 40 (the same values as Python x[1:4]).
```

**Code 2.3:** Inspect the shapes and test both proposed selections.

Expected result, derived from the code and stated model; not captured MATLAB output: the shapes are $1 \times 3$ and $3 \times 1$; proposed A selects $10, 20, 30, 40$, while proposed B selects $20, 30, 40$.

**Selection activity: compare values, not just syntax**

For `x = [10, 20, 30, 40, 50]`, Python `x[1:4]` includes indices 1, 2, and 3. MATLAB `x(2:4)` includes positions 2, 3, and 4. Both select $20, 30, 40$. The proposal `x(1:4)` also includes 10 and is not equivalent.

### 2.4.1. Construct a grid with a stated sample policy

For integer samples 0 through 5, Python uses `range(0, 6)` and MATLAB uses `0:5`. For a step of two starting at zero with an upper bound of five, the resulting values are $0, 2, 4$: neither rule adds a final value that the progression does not reach. For 101 samples from 0 to 10 including both ends, the examples use `np.linspace(0, 10, 101)` and `linspace(0, 10, 101)`.

```python
# Slides 34-36: the second through fourth measurements are 20, 30, 40.
import numpy as np
x = [10, 20, 30, 40, 50]
print(x[1:4])  # Stop index 4 is excluded.
print(list(range(0, 6)))  # Six integer sample values: 0 through 5.
print(list(range(0, 6, 2)))  # The step reaches 4, not 5.
grid = np.linspace(0.0, 10.0, 101)  # Endpoint and count are explicit.
print(len(grid), grid[0], grid[-1])
```

**Code 2.4:** Check selected values, integer grids, and an explicit floating-grid count.

```
[20, 30, 40]
[0, 1, 2, 3, 4, 5]
[0, 2, 4]
101 0.0 10.0
```

```matlab
% Source slide 36: inspect the endpoint policy rather than assuming it.
disp(0:5);                  % Six integer values, including 5.
disp(0:2:5);                % The step reaches 4; it does not reach 5.
grid = linspace(0, 10, 101);  % Explicit count with both endpoints included.
fprintf('%d %.1f %.1f\n', numel(grid), grid(1), grid(end));
```

**Code 2.5:** The matching MATLAB endpoint and count checks.

Expected result, derived from the code and stated model; not captured MATLAB output: $0, 1, 2, 3, 4, 5$; then $0, 2, 4$; the final report has 101 points with endpoints 0 and 10.

For a numerical simulation, an extra endpoint may mean an extra update. Inspect the actual length and endpoint before trusting downstream results. The source uses `numel` for a MATLAB vector. Do not generalize the Python list `len` rule to mean total element count for every multidimensional array; NumPy's length and shape are different properties.[5]

## 2.5. Elementwise and matrix operations ask different questions

*Lecture-slide alignment: Week 13 source PDF, pp. 37–38.*

Suppose three forces belong to three areas in the same order. With force in N and area in $mm^2$, each corresponding ratio is already in MPa. MATLAB `./` performs the required element-by-element division. Plain `/` on nonscalar inputs asks a matrix-division question instead; it is not a shorter spelling of the same operation.

```m
% Source slides 37-38: corresponding entries versus matrix operations.
force_n = [1000.0 2000.0 3000.0];
area_mm2 = [100.0 100.0 100.0];
stress_mpa = force_n ./ area_mm2;  % Pairwise division, giving 10, 20, 30 MPa.
disp(stress_mpa);
A = [1 2; 3 4];
entry_squares = A .^ 2;  % Square each entry independently.
matrix_square = A ^ 2;  % Multiply A by A using matrix multiplication.
disp(entry_squares);
disp(matrix_square);
```

**Code 2.6:** Pairwise stress calculations and two meanings of squaring.

Expected result, derived from the code and stated model; not captured MATLAB output: stress values $10, 20, 30$ MPa; the two matrix results are shown below.

For the source matrix,

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \qquad \text{square_each}(A) = \begin{bmatrix} 1 & 4 \\ 9 & 16 \end{bmatrix}, \qquad A^2 = AA = \begin{bmatrix} 7 & 10 \\ 15 & 22 \end{bmatrix}.$$

The first result squares each entry independently. The second multiplies rows by columns: its upper-left entry is $1 \cdot 1 + 2 \cdot 3 = 7$, and its upper-right entry is $1 \cdot 2 + 2 \cdot 4 = 10$. The other entries follow the same rule. In MATLAB source, write these two expressions as `A .^ 2` and `A ^ 2`.

The NumPy equivalents are `A ** 2` for elementwise squares and `A @ A` for the matrix product. These conventions are not interchangeable with ordinary Python lists. The following added counterpart uses precisely the source values to make the comparison executable in Python.

```python
# Python counterparts of the MATLAB arrays on slides 37-39.
import numpy as np
force_n = np.array([1000.0, 2000.0, 3000.0])
area_mm2 = np.array([100.0, 100.0, 100.0])
print(force_n / area_mm2)  # N/mm^2 equals MPa, element by element.
A = np.array([[1, 2], [3, 4]])
print(A ** 2)  # Square each entry.
print(A @ A)  # Matrix product: each entry is a row-column dot product.
temperature_c = np.array([19.5, 20.0, 25.0, 30.0, 30.5])
mask = (temperature_c >= 20.0) & (temperature_c <= 30.0)
print(temperature_c[mask])  # Preserve the order of selected readings.
```

**Code 2.7:** Worked Python counterpart: pairing, matrix multiplication, and masking.

```
[10. 20. 30.]
[[ 1  4]
 [ 9 16]]
[[ 7 10]
 [15 22]]
[20. 25. 30.]
```

Example provenance: Python counterpart added for the MATLAB source examples on pages 37–39; array values and boundary rules are unchanged.

**Common mistake**

Do not use a dot before every operator indiscriminately. MATLAB array and matrix addition are the same for compatible operands; `.+` is not needed. The multiplication and power questions are genuinely different. For the source's ordinary scalar *gt* calculation, multiplying a scalar by a vector is a defined operation without changing the physical model.[6]

## 2.6. Masks and text comparisons

*Lecture-slide alignment: Week 13 source PDF, pp. 39–40.*

A **logical mask** has a true or false entry corresponding to each data value. The source makes two elementwise comparisons, combines them with `&`, and uses the resulting mask to retain temperatures inside the inclusive interval.

```
% Source slide 39: combine elementwise comparisons, not scalar questions.
temperature_c = [19.5 20.0 25.0 30.0 30.5];
mask = temperature_c >= 20.0 & temperature_c <= 30.0;
within_c = temperature_c(mask);  % Preserve values in acquisition order.
disp(within_c);  % Expected values: 20, 25, 30.
fprintf('%d\n', sum(mask));  % Three true entries.
```

**Code 2.8:** Use an elementwise mask to retain the three normal readings.

Expected result, derived from the code and stated model; not captured MATLAB output: the mask is false, true, true, true, false; selected values are $20, 25, 30$; its true-entry count is 3.

The single `&` and `|` operators combine elementwise logical results here. MATLAB `&&` and `||` serve short-circuit scalar conditions. In the matching NumPy expression, each comparison is explicitly parenthesized before combining the masks; the comparisons are not Python's scalar `and` operation.

### 2.6.1. Choose an interface that compares the whole text

The source Python expression `unit == "kPa"` compares string content. Its MATLAB counterpart deliberately uses a character vector `'kPa'` and `strcmp`. MATLAB also has string arrays; these notes do not silently switch between the two representations. The question is whether the whole stored label matches the required label, not whether a subset of characters matches.

The unit comparison is case-sensitive as written. Accepting several user spellings would be a separate input-normalization rule. It is not legitimate to change a scientific unit label or asset identifier without considering whether case or punctuation carries meaning.

## 2.7. Conditions and loops retain their contracts

*Lecture-slide alignment: Week 13 source PDF, pp. 41–42.*

Both source implementations classify FoS $= 2.0$ as within the stated rule because the condition is FoS $\geq 2.0$. Replacing `>=` with `>` changes the accepted boundary. MATLAB terminates the conditional block with `end`; Python uses indentation. The purpose of the branch remains the same.

```m
% Source slides 40-42: the text is a character vector in this example.
unit = 'kPa';
matched = strcmp(unit, 'kPa');  % Compare the complete text content.
fprintf('%d\n', matched);
fos = 2.0;
if fos >= 2.0  % Equality passes the same stated rule as in Python.
    status = 'within stated rule';
else
    status = 'below stated rule';
end
disp(status);
for time_s = 0:5  % The same six sample times as range(0, 6).
    velocity_m_s = 9.81 * time_s;
    fprintf('%d %.2f\n', time_s, velocity_m_s);
end
```

**Code 2.9:** Text content, an inclusive classification, and six scalar loop calculations.

Expected result, derived from the code and stated model; not captured MATLAB output: the equality check is true, the status is "within stated rule", and the six time–velocity pairs are listed in the table below.

For $v(t) = 9.81t$, the loop uses six scalar sample times. The model starts at rest and assumes constant acceleration. It is a teaching calculation rather than a claim about an arbitrary real falling object.

| Time (s) | Velocity (m/s) |
|---|---|
| 0 | 0.00 |
| 1 | 9.81 |
| 2 | 19.62 |
| 3 | 29.43 |
| 4 | 39.24 |
| 5 | 49.05 |

The source Python version may print a different number of decimal digits than the MATLAB `fprintf` report. Different display formatting is not automatically a numerical disagreement. Compare the quantities and a declared tolerance where appropriate, not only text strings from differently formatted reports.

## 2.8. Translate functions and their tests together

*Lecture-slide alignment: Week 13 source PDF, pp. 43–44.*

The function name alone is not the contract. In this example, `count_above` must count values *strictly* above the scalar threshold, including zero matches for an empty input. It assumes the input values are finite.

```python
# Recovered Week 13 companion definition; see the stated input contract.
def count_above(values, limit):
    """Count values strictly above a limit; assume finite numbers."""
    count = 0
    for value in values:
        if value > limit:  # Equality is not a warning.
            count += 1
    return count
```

**Code 2.10:** Recovered Python strict-count definition.

```matlab
function count = count_above(values, limit)
% Source slide 43: finite numeric vector and scalar limit are assumed.
count = 0;  % Empty input therefore has zero matches.
for i = 1:numel(values)
    if values(i) > limit  % Strictly above; equality is not a warning.
        count = count + 1;
    end
end
% The named output variable count is returned when the function ends.
end
```

**Code 2.11:** MATLAB names the output variable in the declaration.

In Python, `return count` sends the count back. In MATLAB, the declaration names `count` as an output, and the function returns its final value when the function ends. Both initialize the counter once before the loop. Both increment it only when `>` is true. The final MATLAB `end` closes the function, distinct from the `end` lines closing the conditional and loop.

Save the primary MATLAB function in `count_above.m`, and call it from the same folder or an appropriate MATLAB search path. A missing-file or path problem is different from an incorrect comparison. The wrapper and filename matter when moving from a slide excerpt to a runnable program.

### 2.8.1. Complete the displayed stress contract explicitly

The slides reference a full MATLAB `stress_mpa` function but do not print its complete implementation. The following is a *worked completion* for these notes, not a claim that the historical source file was recovered.

```m
function result = stress_mpa(force_n, area_mm2)
% Worked completion of the finite-real-scalar contract on slides 21 and 44.
% The original complete MATLAB function was not recovered with the slides.
if ~isnumeric(force_n) || ~isreal(force_n) || ~isscalar(force_n) ...
        || ~isfinite(force_n)
    error('ENG101:InvalidForce', 'Force must be a finite real scalar.');
end
if ~isnumeric(area_mm2) || ~isreal(area_mm2) || ~isscalar(area_mm2) ...
        || ~isfinite(area_mm2) || area_mm2 <= 0
    error('ENG101:InvalidArea', 'Area must be finite and positive.');
end
% Convert numeric inputs to double before division; N/mm^2 equals MPa.
result = double(force_n) / double(area_mm2);
if ~isfinite(result)
    error('ENG101:InvalidResult', 'The result must be finite.');
end
end
```

**Code 2.12:** Worked completion: validate real scalar inputs before calculating stress.

Example provenance: new implementation of the contract stated on source pages 21 and 44.

The tests separate a legitimate zero force from an invalid zero area. The finite result check addresses output representability; it is not a physical-design criterion. The explicit conversion to `double` avoids performing the requested ratio using an unintended integer numeric representation.

**Stop, predict, and try it**

Predict the results of these MATLAB calls before running them:

1. `count_above([], 30);`
2. `count_above([29.5 30 30.5], 30);`
3. `stress_mpa(10000, 500).`

Then decide what must happen for area zero. The expected results are 0, 1, 20 MPa, and rejection. These answers follow from the contracts.

The complete `examples/matlab/run_checks.m` is supplied for a native MATLAB run. It includes checks of the boundary, signed force, rejection before division, cooling sample counts, and matrix operations. It has not been executed in MATLAB here. This preserves the evidence distinction made by the source slide rather than substituting a Python calculation for a MATLAB test.

## 2.9. A plot has a data and model contract

*Lecture-slide alignment: Week 13 source PDF, p. 45.*

The two plotting programs use 101 points from 0 to 10 seconds and compute $v = 9.81t$. The horizontal axis is time in seconds; the vertical axis is velocity in metres per second. The numerical model, units, sample count, and endpoint policy must agree before the plots are compared.

```python
# Complete imports for the plotting excerpt on slide 45.
import numpy as np
import matplotlib.pyplot as plt
time_s = np.linspace(0.0, 10.0, 101)  # Include both endpoints.
velocity_m_s = 9.81 * time_s  # Constant-acceleration teaching model.
plt.plot(time_s, velocity_m_s)
plt.xlabel("Time (s)")
plt.ylabel("Velocity (m/s)")
plt.title("Constant-acceleration model")
plt.grid(True)  # Aid numerical reading; this is not experimental evidence.
plt.show()  # Display interactively when a graphical backend is available.
```

**Code 2.13:** Complete Python plotting example, including the required imports.

```m
% Source slide 45: matching axes, units, model, and sample count.
time_s = linspace(0.0, 10.0, 101);
velocity_m_s = 9.81 * time_s;  % Scalar-times-vector multiplication.
plot(time_s, velocity_m_s);
xlabel('Time (s)');
ylabel('Velocity (m/s)');
title('Constant-acceleration model');
grid on;  % These points represent a model, not measured velocity data.
```

**Code 2.14:** MATLAB counterpart with the same axis meanings and sample count.

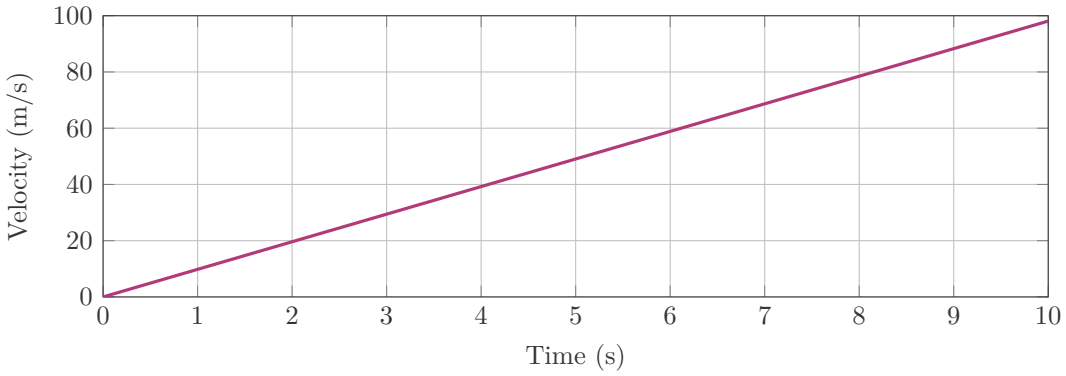



**Figure 2.2.:** Calculated constant-acceleration curve for 101 points, initially at rest.

Source basis: source page 45; calculated model curve redrawn from $v = 9.81t$. Diagram redrawn for these notes.

The line is not a measured trajectory. A visually plausible graph does not test failure handling, data shape, or every array position. Inspect a small known value, the sample count, and the endpoints as well. In the Python example, `np` and `plt` are aliases supplied by imports; they are not built-in names.

## 2.10. Cooling: preserve the discrete update

*Lecture-slide alignment: Week 13 source PDF, pp. 46–48.*

The source cooling model uses an initial temperature of 100 °C, an ambient temperature of 25 °C, a cooling constant $k = 0.05\,\mathrm{s}^{-1}$, and a time step $\Delta t = 1\,\mathrm{s}$. One hundred updates advance the

model to 100 seconds. The update is

$$T_{i+1} = T_i - \Delta t\, k(T_i - T_{\text{ambient}}). \tag{2.1}$$

The quantity $k\Delta t$ is dimensionless, so the subtraction has temperature units. Since the initial temperature exceeds the ambient value, the first change is negative:

$$T_1 = 100 - 1(0.05)(100 - 25) = 96.25\,^{\circ}\text{C}.$$

That sign and scale check is an independent starting test.

### 2.10.1. Python stores the initial state, then appends updates

```python
# Recovered Week 13 companion definition; see the stated input contract.
def cooling_euler(n_steps=100, dt_s=1.0):
    """Return sample times and temperatures for the stated model."""
    if type(n_steps) is not int or n_steps < 0:
        raise ValueError("n_steps must be a nonnegative integer")
    if not math.isfinite(dt_s) or dt_s <= 0:
        raise ValueError("dt_s must be finite and positive")
    times_s = [0.0]
    temperatures_c = [100.0]
    # Fixed model: ambient = 25 deg C; cooling constant = 0.05 /s.
    for step in range(n_steps):
        old_c = temperatures_c[-1]
        new_c = old_c - 0.05 * (old_c - 25.0) * dt_s
        times_s.append((step + 1) * dt_s)
        temperatures_c.append(new_c)
    return times_s, temperatures_c
```

**Code 2.15:** Recovered full Python function behind the source's core cooling excerpt.

The initial lists contain time zero and the initial temperature. Each iteration reads the latest temperature with `[-1]`, computes one new value, appends its physical time, and appends the new temperature. `range(n_steps)` produces exactly `n_steps` iterations, not `n_steps + 1`. The extra stored sample is the initial condition.

The function validates the update count and positive finite step. It does not validate that a selected step is sufficiently accurate for every purpose. Argument validation, numerical resolution, and physical-model suitability are separate questions.

### 2.10.2. MATLAB allocates the same sequence in advance

```
function [times_s, temperatures_c] = cooling_euler(n_steps, dt_s)
% Worked wrapper for the core recurrence printed on source slide 48.
% Assumptions: integer n_steps >= 0 and finite scalar dt_s > 0.
times_s = (0:n_steps) * dt_s;  % Include the initial time.
temperatures_c = zeros(size(times_s));  % Allocate every output position.
temperatures_c(1) = 100.0;  % Initial condition, before any update.
for i = 1:n_steps
    old_c = temperatures_c(i);
    % Ambient = 25 deg C; cooling constant = 0.05 per second.
    temperatures_c(i + 1) = old_c - 0.05 * (old_c - 25.0) * dt_s;
end
end
```

**Code 2.16:** Worked callable wrapper around the MATLAB recurrence shown on slide 48.

Example provenance: the source recurrence is retained; a function declaration and explicit caller assumptions complete the displayed core excerpt.

The time vector has `n_steps + 1` entries, and `zeros(size(times_s))` allocates a temperature array of that shape. Position 1 is overwritten with the initial condition. Each subsequent assignment writes position `i + 1`; it does not overwrite the previous state.

| Physical state | Time | Python position | MATLAB position |
|---|---|---|---|
| Initial state | 0 s | 0 | 1 |
| After first update | 1 s | 1 | 2 |
| After 100 updates | 100 s | 100 | 101 |

**Common mistake**

Adding one to an index is not the same as adding an update. A translated loop that performs 101 updates would represent a different final time. Match physical sample labels first, then translate their positions.

## 2.11. Agreement, discretization error, and physical evidence

*Lecture-slide alignment: Week 13 source PDF, pp. 49–50.*

For this fixed step, subtracting the ambient temperature from Equation (2.1) gives

$$T_{i+1} - 25 = (1 - 0.05)(T_i - 25) = 0.95(T_i - 25).$$

Repeated substitution therefore gives the exact mathematical solution of the *discrete recurrence*,

$$T_i = 25 + 75(0.95)^i.$$

This expression is an independent check of the loop, up to floating-point roundoff. The source also gives the continuous cooling-model solution

$$T(t) = 25 + 75e^{-0.05t}.$$

At 100 seconds, the two expressions are not equal. They answer slightly different numerical questions: one evolves a discrete Euler update and the other evaluates the continuous-model solution.

```python
# Slides 46-50: run the recovered recurrence, then check independent answers.
import math
from models import cooling_euler
times_s, temperatures_c = cooling_euler(n_steps=100, dt_s=1.0)
discrete_c = 25.0 + 75.0 * 0.95 ** 100  # Closed form of the recurrence.
exact_c = 25.0 + 75.0 * math.exp(-5.0)  # Continuous-model solution.
assert len(times_s) == len(temperatures_c) == 101
assert temperatures_c[1] == 96.25  # First update derived by hand.
assert abs(temperatures_c[-1] - discrete_c) < 1e-12
print("Samples:", len(times_s))
print(f"First update: {temperatures_c[1]:.2f} deg C")
print(f"Euler at 100 s: {temperatures_c[-1]:.6f} deg C")
print(f"Continuous at 100 s: {exact_c:.6f} deg C")
print(f"Absolute difference: {abs(exact_c - temperatures_c[-1]):.6f} deg C")
```

**Code 2.17:** Check the sample contract, recurrence, and continuous-model reference separately.

```
Samples: 101
First update: 96.25 deg C
Euler at 100 s: 25.444040 deg C
Continuous at 100 s: 25.505346 deg C
Absolute difference: 0.061306 deg C
```

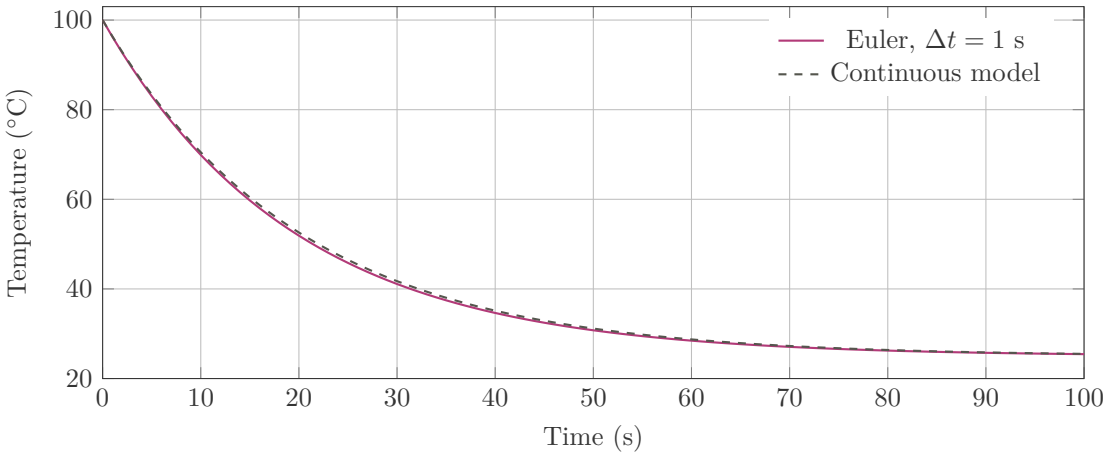



**Figure 2.3.:** Regenerated model curves. Their agreement is not experimental validation or evidence of a native MATLAB run.

Source basis: source pages 49–50; regenerated from the recovered Python recurrence and stated continuous formula, not recovered historical CSV or native MATLAB output. Diagram redrawn for these notes.

The two close curves should not be interpreted as two independent experiments. Both arise from the same stated model. A near match between Python and MATLAB would establish consistency of their discrete implementations, whereas comparison to the exponential solution reveals time-discretization error. Neither comparison proves that the model describes an actual sensor or cooling device accurately.

### 2.11.1. A smaller step must preserve the duration

The source suggests a resolution check. In the following worked extension, the final physical time remains 100 seconds while the step is reduced to 0.5 and 0.25 seconds. The numbers of updates must therefore increase to 200 and 400.

```python
# Worked completion of slide 50: retain the same physical end time.
import math
from models import cooling_euler
exact_c = 25.0 + 75.0 * math.exp(-5.0)
for dt_s in [1.0, 0.5, 0.25]:
    steps = int(100.0 / dt_s)  # These three steps divide 100 s exactly.
    times, temperatures = cooling_euler(steps, dt_s)
    error_c = abs(temperatures[-1] - exact_c)
    print(f"dt={dt_s:.2f} s; updates={steps}; error={error_c:.6f} deg C")
```

**Code 2.18:** Worked extension: compare three resolutions at the same final time.

```
dt=1.00 s; updates=100; error=0.061306 deg C
dt=0.50 s; updates=200; error=0.031121 deg C
dt=0.25 s; updates=400; error=0.015677 deg C
```

Example provenance: added implementation of the resolution check proposed on source page 50.

For these settings the error decreases. This is numerical evidence about the model's approximation, not new hardware evidence. The supplied MATLAB `cooling_demo.m` offers a matching native plotting driver; it is not counted as locally executed.

## 2.12. Debug a mixed-language translation one defect at a time

*Lecture-slide alignment: Week 13 source PDF, pp. 51–52.*

The source deliberately supplies a MATLAB program containing three translation mistakes. It aims to sample $h(t) = 100 - \frac{1}{2}(9.81)t^2$ at $t = 0, 1, 2, 3$ seconds and display the first height.

```python
% INTENTIONALLY WRONG: source slide 51, kept for diagnosis only.
time_s = [0 1 2 3];
height_m = 100 - 0.5 * 9.81 * time_s^2;  % Wrong array operation.
first_height = height_m(0);  % Wrong index: positions start at 1.
print(first_height);  % Wrong output tool for displaying this scalar.
```

**Code 2.19:** Intentional defects: this is a diagnosis exercise, not reference code.

The first defect is using matrix power on a row vector where elementwise powers are required. After that is corrected, the attempted index zero is still invalid in this MATLAB vector. After the index is corrected, `print` is still not the scalar text-display operation intended here. Execution stops at the first failure; it does not necessarily report all three defects in one run.

```python
% Worked repair of the three defects in source slide 51.
time_s = [0 1 2 3];
height_m = 100 - 0.5 * 9.81 * time_s.^2;  % Square each time value.
first_height = height_m(1);  % First sample corresponds to t = 0 s.
disp(first_height);  % Expected value: 100 m.
```

**Code 2.20:** Worked repair preserving the model and the four specified sample times.

Expected result, derived from the code and stated model; not captured MATLAB output: the first displayed height is 100 m. The complete calculated array is $100, 95.095, 80.38, 55.855$ m.

**Answer and explanation**

The smallest semantic repairs are `time_s.^2`, `height_m(1)`, and `disp(first_height)`. A correct repair changes the target-language operation, index, and output call without changing gravity, the initial height, or the sample times. These expected values were derived independently; no native MATLAB execution is claimed.

A useful MATLAB review checks quantity and unit names, one-based positions, colon endpoints, operand shapes, the intended array or matrix operation, matching block endings, and known-answer, boundary, and failure tests. Code acceptance alone does not establish selection of the right data.

## 2.13. Lecture 38 summary and exit-ticket answers

*Lecture-slide alignment: Week 13 source PDF, p. 53.*

**Section summary**

Start from the object type and shape. Translate physical positions, not written index numbers. Distinguish elementwise operations from matrix operations. Carry the original boundary rules and tests into functions and loops. For simulations, compare sample counts, discrete updates, numerical resolution, and model interpretation separately.

**Translate `x[1:4]`.** For the same five-element vector, MATLAB `x(2:4)` selects the same second through fourth values.

**Explain `A.^2` and `A^2`.** The first squares each entry; the second is matrix squaring for this square matrix. The source matrix produces $\left[\begin{smallmatrix} 1 & 4 \\ 9 & 16 \end{smallmatrix}\right]$ and $\left[\begin{smallmatrix} 7 & 10 \\ 15 & 22 \end{smallmatrix}\right]$, respectively.

**Where is the initial cooling temperature?** In MATLAB position 1, corresponding to time zero and Python index 0.

**Why is the Euler result not exactly the continuous result?** The recurrence approximates the continuous rate over finite steps. Even correctly translated implementations retain that discretization error. More output digits do not remove it.

# 3 C++ and Java: Read, Translate, and Test

*Lecture-slide alignment: Week 13 source PDF, pp. 54–78; Lecture 39.*

**Learning outcomes**

After studying this chapter, you should be able to locate a familiar calculation inside a C++ or Java program; recognize declared types, blocks, loops, and callable interfaces; distinguish numeric-vector copying from reference aliasing; connect class state and validated updates to Week 9; and reuse independent tests across implementations.

## 3.1. Read a complete C++ program from the calculation outward

*Lecture-slide alignment: Week 13 source PDF, pp. 55–57.*

The source C++ example calculates current from the same 12 V and 4 Ω used earlier. The additional syntax supplies a program entry point, declarations, and output facilities. It does not change the physical equation.

```cpp
// Source slide 56: a complete C++ program, not a Python statement.
#include <iostream>
int main() {
    double voltage_v = 12.0;  // V
    double resistance_ohm = 4.0;  // ohm; positive resistance is assumed.
    double current_a = voltage_v / resistance_ohm;
    std::cout << "Current = " << current_a << " A\n";
    return 0;  // Successful process exit, not the current itself.
}
```

**Code 3.1:** Complete C++ Ohm's-law program.

```
Current = 3 A
```

`##include <iostream>` supplies declarations for the standard stream output used here. The function `int main()` is the entry point in this ordinary program. Braces group its body. The declarations specify `double` variables, and each ends with a semicolon. `std::cout` is the standard output stream; chained « operations send the label, number, and unit to it. `return 0;` reports successful process completion to the operating system; it does not return the electrical current as the program's engineering result.

### 3.1.1. Compile, then execute

The following are *terminal commands*, not Python statements. They assume a suitable C++ toolchain is already available. Run them in the folder containing `ohms_law.cpp`.

```
# Terminal commands in examples/cpp; a C++ toolchain must already exist.
g++ -std=c++17 -Wall -Wextra ohms_law.cpp -o ohms_law
./ohms_law
```

**Code 3.2:** Compile a C++17 source file and run the produced executable.

The option `-std=c++17` chooses the language standard for the build. `-Wall` and `-Wextra` request useful warnings. The argument after `-o` names the output executable. The next command executes it from the current directory. The source must compile and link successfully before this new executable exists.

For a Windows `g++` toolchain used from PowerShell, the intended variant is:

```
# PowerShell, in examples/cpp with a Windows g++ toolchain installed.
g++ -std=c++17 -Wall -Wextra ohms_law.cpp -o ohms_law.exe
.\ohms_law.exe
```

**Code 3.3:** PowerShell executable naming and invocation.

The executable spelling is clarified without a space in its name. Windows-specific execution is not claimed for this Linux preparation environment.

A compiler diagnostic indicates a problem during building; a runtime exception or incorrect result occurs after a program starts. Warnings deserve inspection even when an executable is produced. Conversely, the existence of an executable does not establish that the formula, unit conversion, or data selection is correct.

## 3.2. Java places the same calculation inside a class

*Lecture-slide alignment: Week 13 source PDF, pp. 58–59.*

```
// Source slide 58: the public class name matches OhmsLaw.java.
public class OhmsLaw {
    public static void main(String[] args) {
        double voltageV = 12.0;  // V
        double resistanceOhm = 4.0;  // ohm; positive resistance is assumed.
        double currentA = voltageV / resistanceOhm;
        System.out.println("Current = " + currentA + " A");
    }  // void means the method returns no value to its caller.
}
```

**Code 3.4:** Java Ohm's-law program with the conventional main method.

```
Current = 3.0 A
```

The public class name `OhmsLaw` matches the filename `OhmsLaw.java`. In this conventional entry method, `public` makes the method accessible, `static` allows it to be called without first constructing an `OhmsLaw` instance, and `void` means the method does not return a value. `String[] args` provides a parameter for command-line text arguments; the simple example does not use them.

`System.out.println` displays the assembled string and finishes the line. Here `+` joins text with the numeric result for reporting. The output may show `3.0`, whereas the C++ stream shows `3`. Both represent the same 3 A result; this is a formatting difference, not a physical difference.

```
# Terminal commands in examples/java; a JDK must already be installed.
javac OhmsLaw.java
java OhmsLaw
```

**Code 3.5:** Compile Java source and launch the named class.

`javac` produces class files containing bytecode. `java OhmsLaw` launches the runtime for the class; the command does not append `.java` or `.class` in this two-step workflow. The source uses conventional syntax rather than requiring a release-specific alternative entry style.[9]

**Key idea**

Identify the familiar inputs and formula first. Then learn only enough surrounding structure to build and execute that calculation. Week 13 is a transfer unit, not a requirement to master all of Java's class machinery or C++'s object model.

## 3.3. Declared types do not settle numerical meaning

*Lecture-slide alignment: Week 13 source PDF, pp. 60–61.*

The earlier division trap remains relevant when operands are variables. With `int samples = 7` and `int groups = 2`, assigning `samples / groups` to a `double` still yields 3.0. Multiplying an operand by `1.0` before division makes this expression use floating arithmetic and retain 3.5. A declaration describes representation and permitted operations, not the engineer's intended ratio.

The source also notes finite integer ranges in C++ and Java. Python integers have different range behavior. These notes do not add an overflow experiment or treat all numeric inputs as interchangeable. The shared examples deliberately use small values, so the comparisons can focus on meaning rather than advanced numeric representations.

### 3.3.1. Chained comparisons do not translate literally

Python supports `20.0 <= temperature_c <= 30.0`. The intended target-language test compares the temperature twice and combines the results with `&&`. Java calls the result type `boolean`; C++ uses `bool`.

In Java, writing the numeric chain literally is rejected. In C++, the first comparison can produce a Boolean that is then compared numerically with 30. For a temperature of 35, the first comparison is true; comparing that result with 30 can misleadingly succeed. A false physical conclusion can therefore come from a syntactically valid expression.

```cpp
// Worked runnable context for source slides 16-20 and 60-61.
#include <iostream>
int main() {
    int value = 3;
    // value = "three";  // Invalid typed assignment: kept as a comment.
    double lost = 7 / 2;  // Integer division happens before assignment.
    double kept = 7.0 / 2.0;
    std::cout << lost << ' ' << kept << '\n';
    std::cout << (3 ^ 2) << ' ' << 3 * 3 << '\n';  // XOR, then square.
    double temperature_c = 35.0;
    // Deliberate semantic defect: (20 <= 35) becomes true, then true <= 30.
    bool wrong = (20.0 <= temperature_c) <= 30.0;
    bool within = temperature_c >= 20.0 && temperature_c <= 30.0;
    std::cout << std::boolalpha << wrong << ' ' << within << '\n';
    return value - 3;  // Return zero; use value so no unused-variable warning.
}
```

**Code 3.6:** Runnable operator comparison including a labelled, deliberately wrong C++ condition.

```
3 3.5
1 9
true false
```

Example provenance: complete driver added around the source expressions. A compiler warning about the deliberate Boolean comparison is expected and is evidence to inspect, not a defect to hide.

```java
// Runnable context for slides 16-20 and 60-61.
public class Operators {
    public static void main(String[] args) {
        int value = 3;
        // value = "three";  // Rejected by the compiler if uncommented.
        double lost = 7 / 2;  // Integer division has already discarded .5.
        double kept = 7.0 / 2.0;
        System.out.println(lost + " " + kept);
        System.out.println((3 ^ 2) + " " + value * value);  // XOR, square.
        double temperatureC = 35.0;
        // boolean bad = 20.0 <= temperatureC <= 30.0; // Invalid in Java.
        boolean within = temperatureC >= 20.0 && temperatureC <= 30.0;
        System.out.println(within);
    }
}
```

**Code 3.7:** Java counterpart: invalid typed statements remain commented out.

```
3.0 3.5
1 9
false
```

The Java invalid assignment and invalid numeric chain are left as comments so the supplied demonstration can execute. They are not silently repaired and presented as valid Java. The preserved correct interval condition returns false for 35, as the original Python rule requires.

**Engineering check**

Test both exact interval endpoints, a value inside, and a value on each side. A test at 25 alone would not reveal every wrong chained-comparison translation. Boundary tests from Week 7 remain important even after moving to a statically typed language.

## 3.4. Loops, functions, and methods retain the same task

*Lecture-slide alignment: Week 13 source PDF, pp. 62–64.*

For readings $[29.5, 30.0, 30.5]$ and a limit of 30, the strict high-count result is one. Only the final comparison succeeds. Python's `for value in values:` and the range-style `for (double value : values)` in the source C++ and Java examples traverse entries in order. The two braces closing a condition and loop are the target-language structural counterparts of Python's indentation changes.

### 3.4.1. The C++ function interface

```cpp
// Source slide 63: finite values and a finite scalar limit are assumed.
inline int count_above(const std::vector<double>& values, double limit) {
    // A const reference reads this vector without copying or modifying it.
    int count = 0;
    for (double value : values) {
        if (value > limit) {  // Strict boundary: equality is excluded.
            ++count;
        }
    }
    return count;
}
```

**Code 3.8:** Source C++ definition: read a vector through a const reference.

The return type is `int`. The first parameter is a reference to a vector of `double` values, marked `const` so the function reads it without modifying it through this interface. The reference avoids making a whole vector copy just to perform the scan. The second parameter is the scalar threshold. The body still visits every entry: a short callable interface does not remove the work counted in Weeks 10–11.

The `inline` keyword is retained because the definition is supplied in a header; it should not be read as a guarantee that the compiler removes all call overhead or changes the algorithm's complexity. Understanding advanced inlining behavior is not part of this unit.

### 3.4.2. The Java method interface

```java
// Source slide 64: non-null finite array and finite limit are assumed.
public static int countAbove(double[] values, double limit) {
    int count = 0;
    for (double value : values) {
        if (value > limit) {  // Strict comparison matches Python.
            ++count;
        }
    }
    return count;
}
```

**Code 3.9:** Source Java definition: a static method reads the numeric array.

The method belongs to the supplied `Week13Models` class. Use the class-qualified call below; no instance of that outer class is needed.

```
Week13Models.countAbove(values, 30.0)
```

The array is passed as a reference value, and this particular method deliberately does not assign to its entries. That non-mutation behavior follows from the body, not from a universal promise that Java methods cannot change arrays.

**Preconditions carried by the source count examples**

The inputs are finite numeric readings and a finite scalar limit; the Java array is non-null. The functions return the number of entries strictly above the limit. They assume the small classroom collection size fits the chosen integer count type. They do not validate every element or convert units automatically.

For an empty valid collection, the initialized count 0 is returned. At the exact threshold, the condition is false. Moving the counter initialization inside the loop or changing `>` to `>=` would change behavior regardless of language.

## 3.5. Collections: positions, lengths, and failure behavior

*Lecture-slide alignment: Week 13 source PDF, p. 65.*

| Collection | First | Last | Size |
|---|---|---|---|
| Python list | `x[0]` | `x[-1]` | `len(x)` |
| C++ vector | `x[0]` | `x.back()` | `x.size()` |
| Java array | `x[0]` | `x[x.length-1]` | `x.length` |
| MATLAB vector | `x(1)` | `x(end)` | `numel(x)` |

Requesting the first or last element requires a nonempty collection. The written length interface also differs: `size()` is a method call on the vector, `length` is an array field in Java, and `numel` is a MATLAB function. These details belong to the target collection's interface.

**Do not use invalid C++ indexing as an exception demonstration**

The original instructor notes distinguish ordinary vector `[]` access from bounds-checked `at()`. Invalid indexing does not have the same guaranteed checked-failure behavior across these interfaces. Keep valid indices in the examples; do not intentionally run out-of-range C++ `[]` access and expect it to behave like Python's `IndexError`.[12]

## 3.6. String content and object identity

*Lecture-slide alignment: Week 13 source PDF, p. 66.*

A Python string equality test compares content in the source examples. A C++ `std::string` equality test also compares text content. Java `==` on the string references in this example asks whether they refer to the same object, whereas `equals` checks string content.

The Java source deliberately constructs `new String("kPa")`. This creates a distinct string object for the demonstration, preventing reuse of a literal from hiding the difference between identity and content. The object is known to be non-null in this small example.

```cpp
// Source slides 66-67: std::string equality and numeric-vector copying.
#include <iostream>
#include <string>
#include <vector>
int main() {
    std::string label = "kPa";
    std::cout << std::boolalpha << (label == "kPa") << '\n';
    std::vector<double> a = {10.0, 20.0};
    std::vector<double> b = a;  // Separate vector of numeric values.
    b[0] = 99.0;
    std::cout << a[0] << '\n';  // The original still starts with 10.
    return 0;
}
```

**Code 3.10:** C++ text content comparison followed by an independent numeric-vector copy.

```
true
10
```

```java
// Source slides 66-67: identity, content, and array-reference assignment.
public class TextCopy {
    public static void main(String[] args) {
        String label = new String("kPa");  // Make a distinct string object.
        System.out.println(label == "kPa");  // false: object identity.
        System.out.println(label.equals("kPa"));  // true: text content.
        double[] a = {10.0, 20.0};
        double[] b = a;  // A second reference to the same numeric array.
        b[0] = 99.0;
        System.out.println(a[0]);  // 99.0: the original is also changed.
    }
}
```

**Code 3.11:** Java text identity and content comparisons followed by array aliasing.

```
false
true
99.0
```

The first C++ result is true because the text matches. Java prints false for identity and true for content. The final numeric results in these programs come from the copying experiment below; they should be interpreted separately from string equality.

## 3.7. The same assignment can preserve or share storage

*Lecture-slide alignment: Week 13 source PDF, p. 67.*

In the source C++ example, `std::vector<double> b = a;` creates a separate numeric vector. Assigning 99 to `b[0]` leaves `a[0]` equal to 10. In the Java example, `double[] b = a;` copies the reference to the existing array, so changing `b[0]` also changes what `a[0]` observes. A Python list assignment `b = a` behaves like that aliasing example.

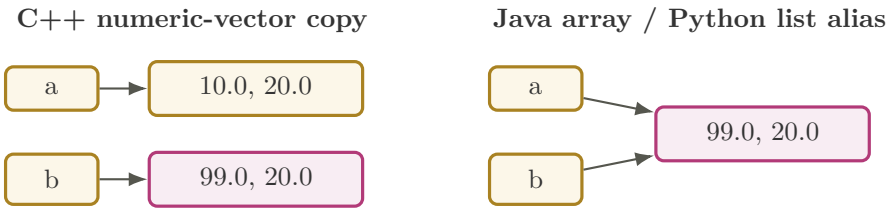



**Figure 3.1.:** After the update through b: separate numeric storage on the left, shared storage on the right.

Source basis: source page 67; separate numeric vectors versus shared array/list references. Diagram redrawn for these notes.

This is an ordinary copy of a C++ vector of numbers, not a claim that every C++ assignment creates a deep independent copy of an arbitrary object graph. Similarly, Java arrays of object references and nested Python lists require further analysis. The teaching comparison intentionally stays with flat numeric collections.

**Stop, predict, and try it**

A calibration function promises to preserve its input readings. What should you test after calling it? Save the original values, inspect the returned or changed output, and confirm that the original collection still matches its saved state. Compiler acceptance alone cannot establish this contract.

## 3.8. Translate a state-change contract

*Lecture-slide alignment: Week 13 source PDF, pp. 68–71.*

The source sensor class is a small record that accepts only finite nonnegative readings. It is not hardware, and its rule is not a statement that all physical sensor quantities must be nonnegative. The program must reject an invalid update before overwriting a previously valid state.

### 3.8.1. Python baseline

```python
# Recovered Week 13 companion definition; see the stated input contract.
class Sensor:
    """A small state model: accept only finite nonnegative readings."""
    def __init__(self, value):
        self.update(value)
    def update(self, value):
        candidate = finite_number(value)
        if candidate < 0.0:
            raise ValueError("reading must be nonnegative")
        self._value = candidate  # Commit only after validation.
    def get_value(self):
        return self._value
```

**Code 3.12:** Recovered Python baseline: validate a candidate before committing it.

The initializer calls the checked update. That method receives a candidate from the recovered `finite_number` helper, rejects a negative value, and only then assigns `self._value`. Its successful return is `None`; the getter returns the stored number. The underscore records a non-public convention, not enforced secrecy.

### 3.8.2. C++ constructor and methods

```cpp
// Source slide 69: a small standalone model of finite nonnegative state.
class Sensor {
private:
    double value_ = 0.0;  // One stored reading per object.
public:
    explicit Sensor(double value) { update(value); }
    void update(double value) {
        if (!std::isfinite(value) || value < 0.0) {
            throw std::invalid_argument("invalid reading");
        }
        value_ = value;  // Commit only after validation succeeds.
    }
    double get_value() const { return value_; }  // Query, do not update.
};
```

**Code 3.13:** C++ source class with private storage and a const query.

The class-name constructor initializes the object; `explicit` is retained as part of this source interface, without adding a new conversion theory unit. `private` restricts ordinary direct source-level access to the field. `public` introduces the callable interface. The query's trailing `const` says that this method does not modify the object through that method.

The complete header includes `<cmath>`, `<stdexcept>`, and `<vector>` for the types and operations used. The class excerpt alone is not a complete executable program. The validation throws `std::invalid_argument`; assignment happens only on the successful path.

### 3.8.3. Java constructor and methods

```java
// Source slide 70: nested class; its value field is still per instance.
public static class Sensor {
    private double value;  // This field is not static.
    public Sensor(double value) { update(value); }
    public void update(double value) {
        if (!Double.isFinite(value) || value < 0.0) {
            throw new IllegalArgumentException("invalid reading");
        }
        this.value = value;  // Commit the local parameter after validation.
    }
    public double getValue() { return value; }  // Return, do not print.
}
```

**Code 3.14:** Java source class nested inside Week13Models.

The `Sensor` type is nested in `Week13Models`. The construction expression is:

`new Week13Models.Sensor(10.0)`

The `static` modifier on this nested class does not make its `value` field shared. That field is an ordinary instance field. The expression `this.value = value` assigns the local parameter to this particular object's field, after validation.

| Role | Python | C++ | Java |
|---|---|---|---|
| Current receiver | `self` | `this` pointer | `this` reference |
| Initialization form | `__init__` | Class-name constructor | Class-name constructor |
| Internal-state notation | `_value` convention | Private member | Private field |
| Inheritance recognition | `class B(A):` | `class B : public A` | `class B extends A` |

Recognition aid from source page 71, not a claim of identical dispatch, lifetime, access, or copying rules.

The key transferable idea is the public state contract. Python's naming conventions, C++ access and method rules, and Java access and reference rules must still be understood on their own terms. Private fields are source-level access controls, not encryption or a physical data-security mechanism.

## 3.9. Test rejection and the surviving state separately

*Lecture-slide alignment: Week 13 source PDF, pp. 72–73.*

```python
# Python comparison for slide 67: assignment aliases the same list.
a = [10.0, 20.0]
b = a
b[0] = 99.0
print(a[0])  # Both names refer to the changed object.
# Slide 72: a rejected proposal is not a replacement measurement.
from models import Sensor
sensor = Sensor(10.0)
sensor.update(12.0)
try:
    sensor.update(-1.0)
except ValueError:
    print("Rejected")
print(sensor.get_value())  # The last valid value is still 12.0.
```

**Code 3.15:** Two reference questions: a list alias and a rejected sensor update.

```
99.0
Rejected
12.0
```

The first line, 99.0, belongs to the list alias experiment. The later lines belong to the sensor state test. The initial reading 10 is updated to 12; the negative proposal is rejected; the next getter must still return 12. Rejecting an input is not enough if an earlier assignment already corrupted the state.

| Operation | Outcome | Stored reading afterward |
|---|---|---|
| Construct with 10 | Accepted | 10 |
| Update to 12 | Accepted | 12 |
| Propose $-1$ | Exception | 12 |
| Call getter | Returns current reading | Still 12 |

The exception class differs: Python uses `ValueError`, C++ uses `std::invalid_argument`, and Java uses `IllegalArgumentException`. These different names do not justify a different state-preservation promise.

### 3.9.1. Complete the shared calculation in each target language

The next definitions are explicitly *worked completions* of the displayed stress contract. The historical complete native modules were not available; these functions supply the missing runnable context rather than purport to reproduce unavailable source files.

```cpp
// Worked completion of the shared contract on source page 21.
inline double stress_mpa(double force_n, double area_mm2) {
    if (!std::isfinite(force_n) || !std::isfinite(area_mm2)
            || area_mm2 <= 0.0) {
        throw std::invalid_argument("invalid stress input");
    }
    double result = force_n / area_mm2;  // N/mm^2 equals MPa.
    if (!std::isfinite(result)) {
        throw std::invalid_argument("nonfinite stress result");
    }
    return result;
}
```

**Code 3.16:** Worked C++ stress implementation for the common finite-input contract.

```java
// Worked completion of the shared contract on source page 21.
public static double stressMpa(double forceN, double areaMm2) {
    if (!Double.isFinite(forceN) || !Double.isFinite(areaMm2)
            || areaMm2 <= 0.0) {
        throw new IllegalArgumentException("invalid stress input");
    }
    double result = forceN / areaMm2;  // N/mm^2 equals MPa.
    if (!Double.isFinite(result)) {
        throw new IllegalArgumentException("nonfinite stress result");
    }
    return result;
}
```

**Code 3.17:** Worked Java stress implementation with the same quantities and units.

Example provenance: new implementations of the source page 21 contract; tested on the common small finite-numeric domain.

The new Python, C++, and Java drivers all print the following shared report under that domain:

```
stress=20.000000
above=1
mean=25.000000
negative=rejected
state=12.000000
```

This compares known results and failure behavior, not merely source strings. Zero-area rejection and a valid zero force are checked separately in the verification suite.

**Historical counts versus this package's evidence**

Source page 73 reports 20 Python test methods, 27 C++ checks, 29 Java checks, and 15 shared-output comparisons for the earlier slide preparation. The underlying historical test files were not recovered, so those counts are not asserted as newly reproduced. The reading package includes a new test suite and a separate machine-readable execution report. MATLAB remains outside the locally executed set.

## 3.10. Exchange data through a documented interface

*Lecture-slide alignment: Week 13 source PDF, p. 74.*

The source diagram presents a calculation or device interface in one language writing a documented file that Python can inspect and report. It is an architectural sketch, not a demonstration of an attached physical instrument. Its example schema is

`time_s,temperature_c,status`.

Names, units, ordering, decimal notation, and missing or invalid-value conventions all need agreement. A CSV extension does not enforce them. A correct exporter paired with a reader that interprets the wrong column or unit can still produce a wrong result.

```python
# Worked completion of the schema on slide 74; use an in-memory text file.
import csv
import io
sample =
    io.StringIO("time_s,temperature_c,status\n0.0,25.0,normal\n1.0,31.0,high\n")
for record in csv.DictReader(sample):
    # CSV fields start as text. Units are declared by the column names.
    time_s = float(record["time_s"])
    temperature_c = float(record["temperature_c"])
    print(time_s, temperature_c, record["status"])
```

**Code 3.18:** Worked completion: read the displayed schema from an in-memory CSV fixture.

```
0.0 25.0 normal
1.0 31.0 high
```

Example provenance: added small fixture illustrating the source schema; not a complete data validator or hardware connection.

The CSV reader returns text fields. `float` converts the two numeric fields explicitly, while the status remains text. The header names communicate units but do not validate that the values were actually measured or exported using them. A proper CSV reader also avoids the assumption that every separator can be handled safely with a naive string split.

## 3.11. Worked activity: transfer a first-threshold search

*Lecture-slide alignment: Week 13 source PDF, p. 75.*

The Python source returns the first zero-based position whose value is strictly greater than the limit, or `None` after all candidates fail.

```python
# This function is transcribed from slide 75; it assumes finite inputs.
def first_above(values, limit):
    """Return the first zero-based index strictly above limit, or None."""
    for index, value in enumerate(values):
        if value > limit:  # Equality is not a match.
            return index  # The earliest match ends the whole function.
    return None  # All candidates were examined without finding a match.
```

**Code 3.19:** The source first-match contract uses None for absence.

A simple C++ or Java teaching interface can instead return an integer and use −1 for absence. This is an explicit change to the *representation* of absence, not to its meaning. Position zero must remain a valid success.

```cpp
// Worked solution to slide 75; small vector length must fit in int.
inline int first_above(const std::vector<double>& values, double limit) {
    for (std::size_t i = 0; i < values.size(); ++i) {
        if (values[i] > limit) {
            return static_cast<int>(i);  // Zero is a valid successful result.
        }
    }
    return -1;  // Chosen absence marker, not a Python negative-index request.
}
```

**Code 3.20:** Worked C++ solution with an explicit -1 absence policy.

```java
// Worked solution to slide 75: -1 is the documented absence result.
public static int firstAbove(double[] values, double limit) {
    for (int i = 0; i < values.length; ++i) {
        if (values[i] > limit) {
            return i;  // Zero is a successful first-position result.
        }
    }
    return -1;  // Do not use this as an array position.
}
```

**Code 3.21:** Worked Java solution preserving the earliest matching position.

Example provenance: activity solutions consistent with the source instructor guidance; small C++ vector indices must fit the chosen int return type.

| Values; limit = 30 | Python | C++ / Java |
|---|---|---|
| `[31, 20]` | 0 | 0 |
| `[30, 31, 35]` | 1 | 1 |
| `[20, 30]` | `None` | −1 |
| Empty collection | `None` | −1 |

The caller tests `result is not None` in the Python interface, or `result >= 0` for the chosen native integer interface. It must not use zero to mean absence. Neither should the −1 marker be passed directly into a collection lookup. In Python negative indices can refer to real entries, while Java array positions must be nonnegative; absence must be handled before indexing.

The implementation returns inside the loop only upon a match. Its final absence return is after the loop. That placement carries the same checked-prefix reasoning used in Week 12. The `count_above` function, by contrast, cannot stop after the first match because it must count all matches.

## 3.12. Review a translation and close the week

*Lecture-slide alignment: Week 13 source PDF, pp. 76–78, 87.*

A translation review compares input meanings, dimensions, units, and ordering; strict versus inclusive comparisons; copying and mutation; and failure behavior. Rerun the original known-answer and boundary tests, and record the new toolchain and the changes accepted. A compiler cannot decide whether an engineering unit label or a duplicate-selection policy matches the requirement.

Python remains useful for readable reference implementations, coordinating tasks, and inspecting results. Another language may fit an existing codebase, hardware interface, numerical environment, or measured resource constraint. No source slide establishes a universal language ranking, and these notes make none.

**Section summary**

A language changes how a solution is expressed. It does not remove responsibility for the model, data, units, algorithm, tests, or interpretation. Use declared types and native tools without assuming that they automatically preserve engineering meaning.

**Name one defect a compiler might miss.** A stress value labelled Pa when it is in MPa; a changed boundary; or an aliasing side effect can all survive compilation.

**Why do the copy examples differ?** Ordinary copying of the C++ vector of numbers creates separate elements. Java array assignment copies a reference to the same array, like the source Python list alias.

**Which tests travel with a ported engineering function?** A hand-derived known answer and an exact boundary are a minimum. Retain invalid-input and non-mutation or state-preservation tests when those are part of the contract. The next course unit is the complete-course review, not a demand to learn a new advanced language stack.

# 4 Optional Connections and Reference Examples

*Lecture-slide alignment: Week 13 source PDF, pp. 79–86.*

The examples in this chapter are optional, as they are in the slides. They connect familiar questions to other interfaces without adding a separate matrix-methods course, database course, or software-installation requirement.

## 4.1. Solve the same pair of linear equations

*Lecture-slide alignment: Week 13 source PDF, p. 79.*

The source system is

$$3x_1 + 2x_2 = 12,$$
$$x_1 + 4x_2 = 10.$$

In matrix form, $A\mathbf{x} = \mathbf{b}$, with

$$A = \begin{bmatrix} 3 & 2 \\ 1 & 4 \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} 12 \\ 10 \end{bmatrix}.$$

The independent solution is $x_1 = 2.8$, $x_2 = 1.8$. Substitution gives $3(2.8) + 2(1.8) = 12$ and $2.8 + 4(1.8) = 10$. This is stronger evidence than treating a solver's output as self-validating.

```python
# Optional slide 79: solve the same two equations without forming an inverse.
import numpy as np
A = np.array([[3.0, 2.0], [1.0, 4.0]])
b = np.array([12.0, 10.0])
x = np.linalg.solve(A, b)
residual = A @ x - b  # Substitute the computed solution into both equations.
print("solution:", x.tolist())
print(f"residual norm: {np.linalg.norm(residual):.3e}")
```

**Code 4.1:** Optional NumPy solution and substitution residual.

```
solution: [2.8000000000000003, 1.7999999999999998]
residual norm: 0.000e+00
```

```octave
% Optional source slide 79: solve A*x = b and inspect the residual.
A = [3.0 2.0; 1.0 4.0];
b = [12.0; 10.0];  % Column vector with the same row count as A.
x = A \ b;  % Solve directly; do not form inv(A).
residual = A * x - b;
disp(x);
fprintf('residual norm: %.3e\n', norm(residual));
```

**Code 4.2:** Optional MATLAB counterpart with a column right-hand side.

Expected result, derived from the code and stated model; not captured MATLAB output: solution components 2.8 and 1.8; the residual is zero in exact arithmetic and may be a tiny floating-point quantity in a native run.

The Python call `np.linalg.solve(A, b)` and MATLAB backslash solve the system without explicitly computing a matrix inverse. The residual $\mathbf{r} = A\mathbf{x} - \mathbf{b}$ asks whether substitution approximately recovers the original right-hand side. A tiny residual supports that check; it is not a complete analysis of conditioning or sensitivity to uncertain measurements.

The Python right-hand side has shape `(2,)` in this example, whereas the MATLAB right-hand side is a $2 \times 1$ column. Both calls represent the same two equations, but their stored shape conventions should not be described as identical. Preserve the mathematical role of the vector while checking each library's interface.

## 4.2. SQL requests a result from structured records

*Lecture-slide alignment: Week 13 source PDF, p. 80.*

The optional source query asks for time and temperature columns from rows whose temperature is strictly above 80, ordered by time.

```sql
-- Source slide 80: retrieve values strictly above 80 in time order.
SELECT time_s, temperature_c
FROM readings
WHERE temperature_c > 80
ORDER BY time_s;
```

**Code 4.3:** Optional source SQL query: filtering and explicit output order.

`SELECT` names the returned columns, `FROM` names the relation, `WHERE` expresses the row condition, and `ORDER BY` requests the output ordering. The query describes the desired result; it does not reproduce Python's loop syntax. The familiar task is still to filter records and preserve a deliberate order.

The slides state that an in-memory Python driver returns `[(1.0, 85.0), (2.0, 82.0)]`, but the complete original driver and its fixture were not recovered. The following **worked reconstruction** uses an explicitly added small fixture to reproduce that stated result. It is not presented as the original hidden dataset.

```python
# Worked reconstruction of the SQL driver referenced, but not printed, on slide 80.
import sqlite3
connection = sqlite3.connect(":memory:")  # No external database or file.
try:
    connection.execute("CREATE TABLE readings (time_s REAL, temperature_c REAL)")
    # Added fixture chosen to reproduce the two rows stated on the slide.
    records = [(2.0, 82.0), (0.0, 75.0), (1.0, 85.0), (3.0, 80.0)]
    connection.executemany("INSERT INTO readings VALUES (?, ?)", records)
    result = connection.execute(
        "SELECT time_s, temperature_c FROM readings "
        "WHERE temperature_c > 80 ORDER BY time_s"
    ).fetchall()
    print(result)  # The threshold is strict; output order is requested explicitly.
finally:
    connection.close()  # Release the database even if an operation fails.
```

**Code 4.4:** Labelled SQL-driver reconstruction using Python's standard library.

```
[(1.0, 85.0), (2.0, 82.0)]
```

The connection exists only in memory and uses no network, account, or credentials. Parameter placeholders supply values separately from SQL syntax. The `finally` block releases the connection even if a statement fails. The deliberately out-of-order fixture shows why the query's ordering clause matters; the record at exactly 80 is excluded by the strict comparison.

## 4.3. C is not another name for C++

*Lecture-slide alignment: Week 13 source PDF, p. 81.*

The source's optional C program has a familiar calculation but a different output interface from the main C++ example.

```c
/* Optional source slide 81: this is C, compiled separately from C++. */
#include <stdio.h>
int main(void) {
    double voltage_v = 12.0;      /* V */
    double resistance_ohm = 4.0; /* ohm; positive resistance assumed */
    double current_a = voltage_v / resistance_ohm;
    printf("Current = %.2f A\n", current_a);
    return 0; /* Successful process exit. */
}
```

**Code 4.5:** Optional C example, compiled separately as C11.

```
Current = 3.00 A
```

The header `<stdio.h>` provides the declared formatted-output interface used here. The function `printf` interprets `%.2f` as a floating-point format and displays the same numerical current to two decimal places. This code is identified and compiled as C, not relabelled C++ merely because its punctuation looks similar.

The input assumption is still $R > 0$. This example does not add input validation, advanced pointer operations, or a C course. The distinction prevents an introductory survey from confusing a C-style example in the course book with a complete description of C++.

## 4.4. Other tools: recognition rather than an installation list

*Lecture-slide alignment: Week 13 source PDF, p. 82.*

| Tool | Possible role identified in the source |
|---|---|
| SQL | Query structured stored records. |
| JavaScript | Browser interfaces and interactive dashboards. |
| R | Statistical analysis and reporting. |
| Julia | Scientific and numerical programming. |
| Shell tools | Automate file and command workflows. |
| LabVIEW | Graphical instrumentation and data acquisition. |

These are example roles, not exclusive capabilities, current market-share claims, or a ranking of languages. JavaScript and Java are distinct languages. A new environment is worth learning when the problem, team, interface, or course requires it, not simply because it exists.

## 4.5. Use the supplied package as a reproducible study resource

*Lecture-slide alignment: Week 13 source PDF, pp. 83–86.*

Start with one small example in its own language folder. Read its assumptions and filename, identify whether it is a complete program or a supporting component, and check the README for its exact command. The Python demonstrations need the local `models.py`; the C++ clients need `week13_models.hpp`; the Java clients need `Week13Models.java`. The independent Python companion in the final download folder is self-contained.

NumPy and Matplotlib are used only for the clearly identified numeric-array and plotting comparisons. Most core demonstrations require Python's standard library only. The optional database example uses `sqlite3`, also from the standard library, rather than an external service.

The reading package's new test command is `python tests/verify_all.py`, run from its project root. It executes available Python examples, builds and checks the supplied native programs, and records the environment and outcomes. The native MATLAB checks are in `examples/matlab/run_checks.m`, to be run in MATLAB from that folder. A test supplied as source is not counted as executed merely because its expected result is known.

**Evidence to report**

Keep source code, input fixtures, comments identifying adaptations, actual execution records, and a description of what was not executed. Original source-page claims, regenerated model data, new verification, and native MATLAB expectations are different kinds of evidence and should remain distinguishable.

# 5 Cumulative Practice and Worked Solutions

Example provenance: the following 12 cumulative questions and solutions are added for self-study, using the slide contracts and datasets. They are not announced examination questions.

Make an independent attempt before reading each solution. Record the input meaning, the expected result, and the reason. Executing a program is useful after that prediction, but it should not replace the explanation.

## 5.1. Practice questions

### Question 1 — Units survive a change of language

A program receives $F = 10000\,\mathrm{N}$ and $A = 500\,\mathrm{mm}^2$, computes $F/A$, and prints "20 Pa". Is the arithmetic incorrect, the label incorrect, or both? Give two consistent repairs. Explain why a declared `double` type does not expose the problem.

### Question 2 — A normal mean with abnormal readings

Using $[19.5, 20, 25, 30, 30.5]$ and inclusive limits 20–30, find the mean, the two-sided warning count, the original warning positions, and the strictly-above-30 count. Explain why a translation that returns only the mean does not preserve the whole workflow.

### Question 3 — Expression type before destination type

In C++ or Java, what is the value stored by `double x = 7 / 2;`? What changes when the expression becomes `1.0 * 7 / 2`? Predict `3 ^ 2` in Python, MATLAB with scalar double literals, C++, and Java. State the intended squaring expression for each.

### Question 4 — Equivalent vector selection

For $[10, 20, 30, 40, 50]$, translate Python `x[1:4]` into MATLAB. List the selected values and compare the result with MATLAB `x(1:4)`. Translate the first and last element accesses without changing their physical positions.

### Question 5 — Shape and matrix meaning

For $A = \left[\begin{smallmatrix} 1 & 2 \\ 3 & 4 \end{smallmatrix}\right]$, calculate the two results requested by MATLAB `A.^2` and `A^2`. Explain why the upper-left entries differ. Name the matching NumPy expressions.

### Question 6 — Masks and exact boundaries

Write the five logical entries for the source temperature interval mask. Which values are selected? For an FoS requirement of at least 2.0, which comparison accepts the exact boundary? Why does changing it during translation change the specification?

### Question 7 — Cooling samples and numerical error

For the source cooling example, calculate the first update, the number of stored samples after 100 updates, and the Python and MATLAB positions of that first update. When the step is reduced from 1 to 0.5 seconds, how many updates preserve a final time of 100 seconds? Distinguish a recurrence check from a continuous-model accuracy check.

### Question 8 — Repair the mixed-language height program

The defective code uses MATLAB `time_s^2`, then `height_m(0)`, then `print(first_height)` for `time_s = [0 1 2 3]`. Repair all three operations without changing the model. What is the first height? Why may only the first defect be observed in the initial run?

### Question 9 — Copy, alias, and content

A C++ numeric vector `b` is copy-constructed from `a`, then `b[0]` becomes 99. In the matching Java example, `b = a` assigns an array reference before the same element update. What does `a[0]` show in each? Separately, why does the source Java `new String("kPa")` example distinguish `==` from `equals`?

### Question 10 — Reject a proposal without corrupting state

A sensor is created with 10, successfully updated to 12, then receives $-1$. Identify the validation location, expected failure, and required getter result. Explain what is wrong with a translated setter that assigns first and checks afterward, even if its caller catches the exception.

### Question 11 — First match is not a count

For a threshold of 30, give the first-match results for $[31, 20]$, $[30, 31, 35]$, $[20, 30]$, and an empty sequence. Use Python's `None` policy and the native $-1$ policy separately. What should the native caller test before using the result as an index?

### Question 12 — A translation report that can be trusted

You translate an engineering calculation into another language. It compiles, produces a plausible graph, and agrees with the original at one interior value. Identify at least five additional things that should appear in the verification report. Explain whether supplying MATLAB test files without running them permits a statement that MATLAB verification passed.

## 5.2. Worked solutions

### Solution 1 — Repair the unit contract, not just the variable name

The numerical division is consistent with N divided by $mm^2$. Its result 20 represents 20 MPa. One repair is to label the result MPa and retain area in $mm^2$. A second is to convert $500\,mm^2 = 0.0005\,m^2$, calculate 20,000,000 Pa, and label it Pa. A numeric type does not encode the unit metadata. See Section 1.6.

### Solution 2 — Retain the local evidence

The sum is 125 and the mean is 25 °C. There are two out-of-range values: 19.5 at index 0 and 30.5 at index 4. There is only one value strictly above 30. A mean alone loses the individual warning evidence and the original positions. The difference between one and two is caused by different requested predicates, not by an inconsistent dataset. See Section 1.3.

### Solution 3 — Choose operands before assignment

The integer expression `7 / 2` is evaluated as 3 in the source C++ and Java examples, then stored as 3.0 in the floating destination. With `1.0 * 7 / 2`, a floating operand participates before division, so 3.5 is retained. For `3 ^ 2`, Python, C++, and Java produce 1 by integer XOR; MATLAB produces 9 for its scalar power. Use Python `3 ** 2`, MATLAB `3 ^ 2`, and C++/Java `3 * 3` to square these scalars. See Sections 1.7 and 3.3.

### Solution 4 — Follow the measurement, not the written number

Python `x[1:4]` selects indices 1, 2, and 3: values 20, 30, and 40. MATLAB `x(2:4)` selects those same values. MATLAB `x(1:4)` includes the extra leading 10. The first and last accesses translate from Python `x[0]` and `x[-1]` to MATLAB `x(1)` and `x(end)`. See Section 2.4.

### Solution 5 — Two operations produce two justified answers

Elementwise squaring gives

$$\begin{bmatrix} 1 & 4 \\ 9 & 16 \end{bmatrix},$$

whereas matrix squaring gives

$$\begin{bmatrix} 7 & 10 \\ 15 & 22 \end{bmatrix}.$$

The upper-left entry is either $1^2$ or the dot product $1 \cdot 1 + 2 \cdot 3$. Both results are valid for their requested operations; neither is a mere display variation. The NumPy expressions are `A ** 2` and `A @ A`. See Section 2.5.

### Solution 6 — Equality belongs in the rule

The mask is false, true, true, true, false, so values 20, 25, and 30 remain. A requirement “at least 2.0” uses `>= 2.0`; it accepts the equality boundary. Replacing it with `> 2.0` rejects that boundary and therefore changes the modelled rule. Keep an exact-boundary regression test when porting. See Sections 2.6 and 2.7.

### Solution 7 — Distinguish updates from samples

The first update is $100-1(0.05)(100-25) = 96.25\,^{\circ}\mathrm{C}$. The initial condition plus 100 updates gives 101 samples. The first updated state is Python index 1 and MATLAB position 2. A 0.5-second step needs 200 updates to retain 100 seconds, giving 201 stored samples. Agreement with $25 + 75(0.95)^{100}$ checks the discrete recurrence for the original step. Agreement with $25 + 75e^{-5}$ asks about the continuous solution, which the finite-step Euler method only approximates. See Sections 2.10–2.11.

### Solution 8 — Repair the first wrong operation, then retest

The repaired operations are `time_s.^2`, `height_m(1)`, and `disp(first_height)`. The first height is 100 m because its time is zero. In the defective program, the matrix-power operation on this row vector fails before later statements are reached. After repairing it, rerun to reveal and verify the remaining defects. This is the systematic debugging workflow from Week 7. See Section 2.12.

### Solution 9 — Separate identity from stored numeric values

The source C++ numeric-vector copy leaves `a[0]` equal to 10. The Java array alias makes `a[0]` equal to 99.0. Python `b = a` for the same list also shares the object. In the Java string example, the deliberately new object is not identical to the literal object, so `==` is false; its text matches, so `equals` is true. Neither finding implies universal deep-copy behavior for nested object structures. See Sections 3.6–3.7.

### Solution 10 — Validation precedes commitment

The setter should check finiteness and nonnegativity before assigning to the stored field. The negative proposal raises `ValueError` in Python. C++ throws `std::invalid_argument`; Java throws `IllegalArgumentException`. The next getter must still return 12. Assigning $-1$ first violates the state guarantee even if an exception is thrown later and caught successfully. Test both the rejection event and the surviving state. See Section 3.9.

### Solution 11 — Make absence explicit

The Python results are $0, 1,$ `None`, `None`. Under the chosen native policy they are $0, 1, -1, -1$. The native caller should test `result >= 0` before indexing; zero must be accepted as a successful first-position result. The absence return belongs after the loop, not after the first unsuccessful comparison. The all-matches counter is a different function with a different stopping obligation. See Section 3.11.

### Solution 12 — Report scope, not only success

A defensible report states input/output units, valid domain, shapes and sample ordering, boundary rules, returned and absence conventions, mutation and failure behavior, independent answers, invalid-input tests, toolchain, commands, and model limitations. A plausible plot may hide a wrong sample or unit multiplier. Agreement at one interior value says little about boundaries or failed updates.

Unexecuted MATLAB checks are *supplied or reviewed*, not passed native tests. Keep derived references, measurements, and native outputs distinct. Readiness means recognizing familiar reasoning in new syntax, tracing a small example, and choosing independent tests—not memorizing every command or completing three new language courses.

# 6 Quick Reference, Glossary, and Study Checklist

## 6.1. Cross-language recognition table

*Lecture-slide alignment: Week 13 source PDF, p. 83.*

| Idea | Python | MATLAB | C++ / Java |
|---|---|---|---|
| First vector element | `x[0]` | `x(1)` | `x[0]` |
| Square a scalar | `x**2` | `x^2` | `x*x` |
| Scalar logical AND | `and` | `&&` | `&&` |
| Scalar logical OR | `or` | `\|\|` | `\|\|` |
| End a block | Dedent | `end` | Closing brace |
| Line comment | `#` | `%` | `//` |
| Numeric approximate equality | `abs(a-b) <= tol` | `abs(a-b) <= tol` | Suitable native absolute-value call and tolerance |
| Text-content comparison in the examples | `a == b` | `strcmp(a,b)` for character vectors | `std::string ==` / Java `equals` |
| Absence in first-match activity | `None` | Not defined by that source activity | $-1$, explicitly chosen in the worked solution |

The table is deliberately limited. Return to the array sections before translating a nonscalar operation. A tolerance needs a purpose and a scale; it is not an arbitrary decoration appended to a comparison. MATLAB string arrays and character vectors are not silently interchangeable, and the C++ and Java text examples use their stated object types.

## 6.2. Glossary

**Algorithm**
: A defined sequence of operations for the requested task. Changing its language does not necessarily change its computational work.

**Bytecode**
: An intermediate instruction representation used in the execution paths described for CPython and Java.

**Compiler**
: A tool translating source into another representation. A successful compile is not a proof of engineering correctness.

**Contract**
: The documented relationship among inputs, units, valid domain, output, and failure or mutation behavior.

**Data schema**
A specification of fields, names, meanings, units, ordering, and missing-value conventions for exchanged records.

**Elementwise operation**
An operation applied to corresponding entries, with compatible shape rules determined by the interface.

**Exact continuous solution**
A mathematical solution of the stated continuous model, such as the exponential cooling curve, distinct from a numerical recurrence.

**Execution evidence**
Recorded behavior from an actual run in an identified environment. Expected results and reviewed code are not the same evidence category.

**Initializer / constructor**
Language-specific mechanisms for establishing initial object state. Their names are similar in purpose, but their full semantics need not coincide.

**Instance state**
Data associated with one object. It must not accidentally become shared merely because a field or class modifier was misunderstood.

**Interface**
The callable or data-access surface a client uses. It includes meaning, not only syntax.

**JDK**
The Java development toolchain used here to compile with `javac` and run with `java`.

**JVM**
The Java virtual machine execution environment in the source's illustrated route.

**Library**
Reusable implementations and documented interfaces; a library is distinct from the core language and the editor.

**Logical mask**
A collection of logical entries selecting corresponding data values.

**Matrix operation**
An operation governed by linear-algebra structure, such as row-column multiplication or solving a linear system.

**Preallocation**
Allocating output storage with its intended shape before filling it, as in the MATLAB cooling example.

**Reference alias**
Another name or stored reference leading to the same object, so a mutation can be observed through both.

**Residual**
The discrepancy after substituting a computed answer into an equation; for the optional linear system it is $A\mathbf{x} - \mathbf{b}$.

**Runtime**
The machinery that executes a program. It is not the same thing as the editor used to write the source.

**Shape**
The dimensions of an array, not merely its total number of entries.

**Static method**
In the Java examples, a method called through its class without constructing an instance of that class.

**Toolchain**
The collection of build and execution tools relevant to an actual run, including compiler options and dependencies.

**Translation equivalence**
Agreement with the same declared computational contract on the stated domain, allowing explicitly documented representation changes.

**Validation**
Explicit checks that an input or proposed state satisfies a chosen rule. Input validation is distinct from validating a real-world physical model.

**Value copy**
In the source C++ example, a new numeric vector holding its own elements, distinct from assignment of a reference to an existing array.

## 6.3. A cross-language review checklist

Start by describing the question and writing one answer that does not depend on the implementation. Then check input types, units, array dimensions, selected indices, endpoint policies, and strictness of comparisons. Inspect the result type, absence representation, mutation, and error path. Confirm that the target program does not discard warnings or overwrite valid state while reporting success.

For numerical models, keep the duration and sample count fixed when comparing implementations. Separate agreement between programs from approximation error and physical validity. For performance, do not infer a speed ranking from the language name; identify the actual implementation and workload.

Retain the sources, fixtures, expected answers, run commands, dependencies, and environment record. State which examples were run, which were reviewed only, which are intentional defects, and which are added completions. A clear limitation is more useful than an unsupported assurance.

**Key idea**

The most transferable outcome of ENG 101 is not a catalogue of syntax. It is the ability to formulate a problem, express its rules precisely, inspect a computation, and explain why its evidence is or is not trustworthy.

# A Source basis, coverage, and verification

## A.1. What was preserved and what was completed

The primary source is Yanwei Wang's *Programming for Engineers: Week 13, Python and Programming Beyond Python*, Fall 2026, 87 physical PDF pages. The supplied slide source and instructor guide establish the lecture titles and teaching sequence. The notes retain their engineering examples and scope; explanatory prose, comments, worked answers, and cumulative exercises develop those examples for independent reading.

The retrieved slide companion supplied the five Python definitions `finite_number`, `stress_mpa`, `count_above`, `Sensor`, and `cooling_euler`. Their executable syntax was checked against the preserved original using Python's abstract syntax tree. Additional comments and drivers do not change those recovered definitions. The `first_above` function is transcribed from source page 75.

The missing complete native-language modules were not assumed to exist. C++ header and Java class wrappers, the missing native stress functions, first-match activity solutions, the MATLAB stress function, SQL fixture, CSV demonstration, and verification programs are labelled worked completions. The notes explicitly distinguish a source excerpt from its reconstructed runnable context. No new benchmark, real sensor dataset, or certified material specification is implied.

**Historical evidence and new evidence are separate**

The source page 73 reports 20 Python test methods, 27 C++ checks, 29 Java checks, and 15 shared-result comparisons from the earlier slide preparation. Those raw historical tests were not recovered. The results below are from *new* tests of this reading package. Likewise, the cooling data were regenerated from the recovered update rule; they are not represented as the missing original CSV file.

## A.2. Execution record for this package

The reading-package verification ran 42 Python test methods, 38 independent C++ checks, and 38 independent Java checks. It executed 18 Python demonstration programs, five C++ demonstration programs, five Java demonstration programs, and one C program. Fifteen shared result comparisons used independently stated expectations. MATLAB was not executed. See the supplied machine-readable verification report for the environment and diagnostics.

The machine-readable `data/verification.json` retains software versions, console output, and compiler diagnostics. The C++ chained-comparison demonstration deliberately contains a legal but misleading comparison; any compiler warning for that line is evidence for the exercise, not an unexplained build defect. The main supported commands are described in the README. All test programs use explicit failure checks where a language may otherwise disable assertions.

**MATLAB was not executed.** Its native source and `examples/matlab/run_checks.m` are supplied for execution in MATLAB. Expected values in the notes are derived and labelled. A passed Python analogue is not a claim that MATLAB accepted or executed the MATLAB file.

Headless execution of the Python plotting example is also distinguished from a graphical desktop test.

## A.3. Slide-to-reading coverage

Every physical source page appears below. Titles follow the recovered instructor guide; the destination is a reading section rather than an assertion that the brief slide text alone contains all the worked explanation. Activities, diagrams, outputs, and cautions are expanded together in the indicated section.

| Slide | Source topic | Reading page |
|---|---|---|
| 1 | Programming for Engineers: Week 13 | 2 |
| 2 | Week 13 at a glance | 2 |
| 3 | Scope: transfer, not three new courses | 2 |
| 4 | Python in Engineering and Transferable Ideas | 6 |
| 5 | Lecture 37 learning goals | 6 |
| 6 | Warm-up: what must stay the same? | 6 |
| 7 | You are not starting from zero | 6 |
| 8 | Language, library, editor, runtime | 7 |
| 9 | A small engineering workflow | 7 |
| 10 | Python example: preserve the warning locations | 8 |
| 11 | A summary is not the whole result | 8 |
| 12 | Python tools occupy different layers | 7 |
| 13 | Read an unfamiliar call through its contract | 7 |
| 14 | Same formula, different surface syntax | 9 |
| 15 | Execution involves more than an editor | 10 |
| 16 | Dynamic and static typing: a first comparison | 11 |
| 17 | A valid numeric type does not enforce units | 11 |
| 18 | Recognize the syntax that changes | 11 |
| 19 | Trap 1: division depends on operand types | 12 |
| 20 | Trap 2: a caret is not always a power | 12 |
| 21 | Translate a contract before translating code | 13 |
| 22 | Compare results and behavior, not just text | 13 |
| 23 | Reproducibility includes the toolchain | 10 |
| 24 | Choose a tool from the constraints | 14 |
| 25 | Activity: defend a tool choice | 14 |
| 26 | AI-assisted translation still needs independent checks | 14 |
| 27 | Learn a new language through a familiar problem | 14 |
| 28 | Lecture 37 exit ticket | 15 |
| 29 | MATLAB for Python Users | 16 |
| 30 | Lecture 38 learning goals | 16 |
| 31 | Warm-up: what kind of object is this? | 16 |
| 32 | Scalar calculations: familiar reasoning | 17 |
| 33 | An array has a shape as well as values | 17 |
| 34 | Translate positions, not written index numbers | 18 |
| 35 | Activity: select the same three measurements | 18 |
| 36 | Create a grid with a stated endpoint policy | 18 |
| 37 | Elementwise division pairs the measurements | 19 |
| 38 | Matrix power and elementwise power differ | 19 |
| 39 | Logical masks retain matching values | 21 |
| 40 | Translate text comparisons deliberately | 21 |

| Slide | Source topic | Reading page |
|---|---|---|
| 41 | Conditionals preserve the boundary rule | 21 |
| 42 | A loop still repeats the same calculation | 21 |
| 43 | Functions expose inputs and an output name | 22 |
| 44 | Tests can be translated before the main program | 22 |
| 45 | Plotting transfers as a sequence of questions | 24 |
| 46 | Cooling: transfer the update rule | 25 |
| 47 | Python: append one new state per update | 25 |
| 48 | MATLAB: preallocate the same state sequence | 25 |
| 49 | A plot checks behavior, not the whole translation | 27 |
| 50 | Agreement and numerical accuracy are different checks | 27 |
| 51 | Debugging clinic: a mixed-language translation | 29 |
| 52 | A MATLAB translation checklist | 29 |
| 53 | Lecture 38 exit ticket | 30 |
| 54 | C++ and Java: Read, Translate, and Test | 31 |
| 55 | Lecture 39 learning goals | 31 |
| 56 | C++: locate the familiar Ohm's-law calculation | 31 |
| 57 | C++: build first, then run | 31 |
| 58 | Java: find the same calculation inside main | 32 |
| 59 | Java: source, class files, and the JVM | 32 |
| 60 | Declared types do not settle numeric meaning | 33 |
| 61 | Do not copy a chained comparison literally | 33 |
| 62 | New loop syntax, the same repeated task | 35 |
| 63 | C++ function: explicit inputs and result type | 35 |
| 64 | Java method: the same strict-count contract | 35 |
| 65 | Choose the corresponding collection deliberately | 36 |
| 66 | String content is not always tested with == | 37 |
| 67 | The same assignment spelling can copy different things | 38 |
| 68 | Python baseline: query and update state | 38 |
| 69 | C++: constructor, methods, and private state | 38 |
| 70 | Java: the same state-change contract | 38 |
| 71 | Relate new syntax to the Week 9 ideas | 38 |
| 72 | A rejected update must preserve the last valid state | 40 |
| 73 | Use one small, independently specified test set | 40 |
| 74 | Exchange data without sharing source syntax | 43 |
| 75 | Activity: translate the first-threshold search | 43 |
| 76 | Review a translation like an engineering change | 45 |
| 77 | Use the tool that makes the evidence strongest | 45 |
| 78 | Lecture 39 exit ticket: what travels with you? | 45 |
| 79 | Optional: solve the same matrix equations | 46 |
| 80 | Optional: SQL expresses a requested result | 47 |
| 81 | Optional: C is a different language from C++ | 48 |
| 82 | Optional: other tools to recognize | 49 |
| 83 | Quick reference: preserve meaning across syntax | 54 |
| 84 | How to use the accompanying example package | 49 |
| 85 | Source basis: course and Python–MATLAB bridge | 57 |
| 86 | Source basis: C++, Java, and verification | 57 |
| 87 | Week 13: keep the reasoning, learn the tool | 45 |

# Bibliography


[1] Y. Wang. *Programming for Engineers: Week 13 — Python and Programming Beyond Python*. ENG 101, School of Engineering, Nazarbayev University, Fall 2026. 87-page source presentation and recovered companion.

[2] Nazarbayev University. *ENG 101 — Programming for Engineers: Course Specification Report*, Session 13. Course alignment as identified in the source presentation.

[3] Y. Wang. *Programming for Engineers with Python*, working course manuscript. Chapter 16 and Appendix I are recommended by the source slides for transferable ideas and the MATLAB bridge. Source provenance is recorded in the preserved slide source map.

[4] Python Software Foundation. *Python Tutorial: An Informal Introduction to Python; Classes*. https://docs.python.org/3/tutorial/introduction.html; https://docs.python.org/3/tutorial/classes.html.

[5] NumPy developers. *NumPy: the absolute basics for beginners*. https://numpy.org/doc/stable/user/absolute_beginners.html.

[6] MathWorks. *Array vs. Matrix Operations*. https://www.mathworks.com/help/matlab/matlab_prog/array-vs-matrix-operations.html.

[7] MathWorks. *Array Indexing; colon*. https://www.mathworks.com/help/matlab/learn_matlab/array-indexing.html; https://www.mathworks.com/help/matlab/ref/double.colon.html.

[8] MathWorks. *function; strcmp; isfinite*. https://www.mathworks.com/help/matlab/ref/function.html; https://www.mathworks.com/help/matlab/ref/strcmp.html; https://www.mathworks.com/help/matlab/ref/isfinite.html.

[9] Oracle / OpenJDK contributors. *Getting Started with Java*. https://dev.java/learn/getting-started/.

[10] Oracle. *Java Language Specification*, Java SE 21, Chapter 15, Expressions. https://docs.oracle.com/javase/specs/jls/se21/html/jls-15.html.

[11] C++ working draft. *Multiplicative operators [expr.mul]*. https://eel.is/c++draft/expr.mul.

[12] C++ working draft. *Class template vector [vector]*. https://eel.is/c++draft/vector.


**Key idea**

A successful translation retains an engineering argument: a clear question, a model, a declared representation, a checkable result, and an honest statement of limitations. Different syntax does not remove that responsibility.